Grado en Ingeniería Informática

Tecnologías de Desarrollo de Software

Patrones de Diseño

(Parte 1. Patrones de Creación)

Facultad de Informática

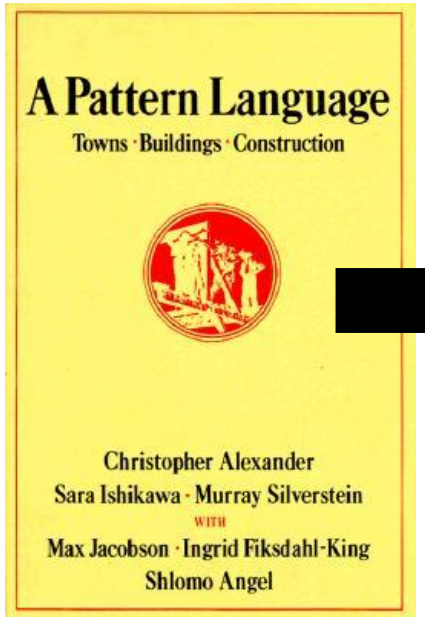
UNIVERSIDAD DE MURCIA

Departamento Informática y Sistemas

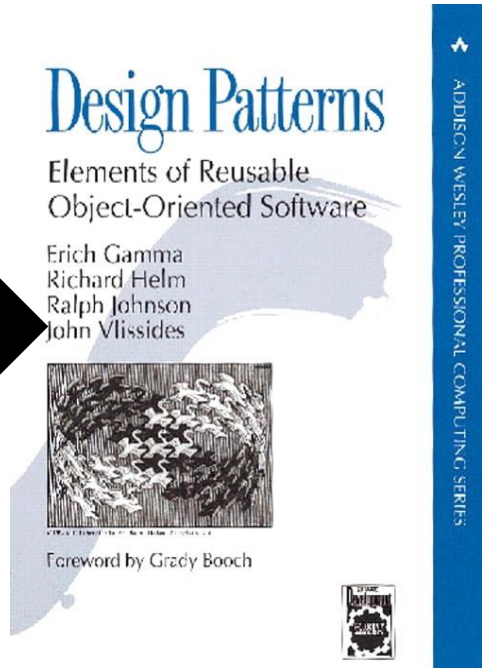
Contenidos

- Introducción a los patrones de diseño
- Patrones de **creación**
  - Factoría Abstracta, *Builder*, Método Factoría, Singleton
- Patrones **estructurales**
  - Adaptador, Composite, Decorador, Fachada, Proxy
- Patrones de **comportamiento**
  - Command, Iterador, Observer, Estado, Estrategia, Método Plantilla

Orígenes

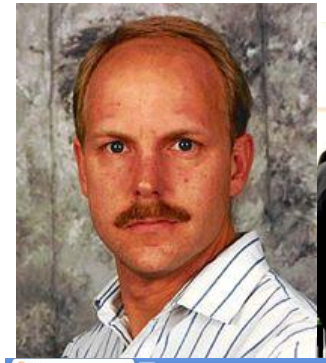


1977



"Design Patterns"
E. Gamma, et al.
Addison-Wesley, **1995**
(Patterns GoF)

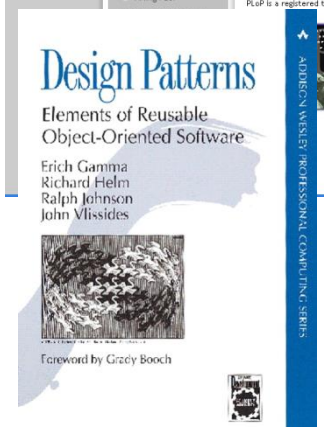
- 1987 – Ward Cunningham and Kent Beck definen un pequeño **lenguaje de patrones** para **Smalltalk** basado en las ideas de Alexander.
- 1991 - Bruce Anderson dirige el primer Patterns Workshop en el OOPSLA.
- 1993 - Kent Beck and Grady Booch organizan el primer encuentro del Hillside Group.
- 1994 – Primera edición de la conferencia PLoP
- 1994 - The Gang of Four (GoF) publican el libro Design Patterns

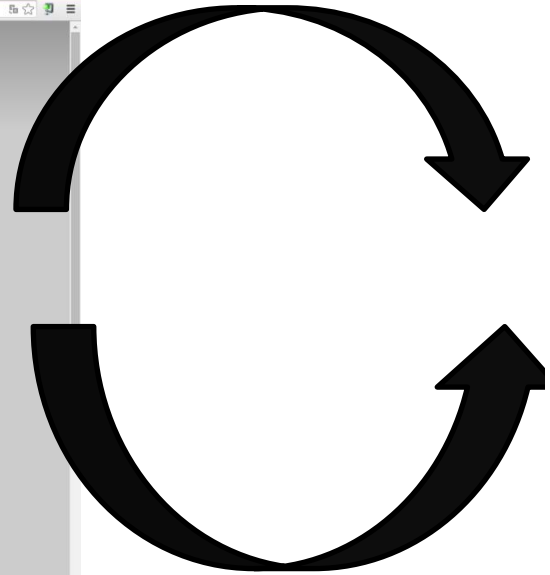




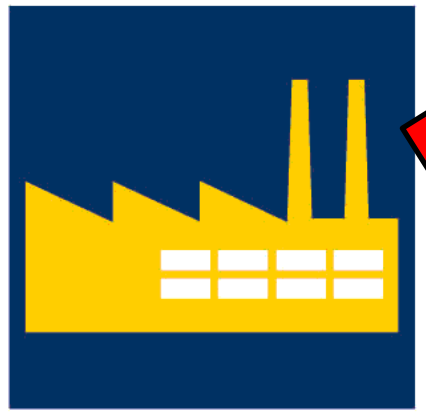


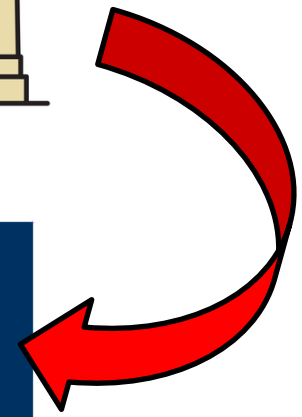








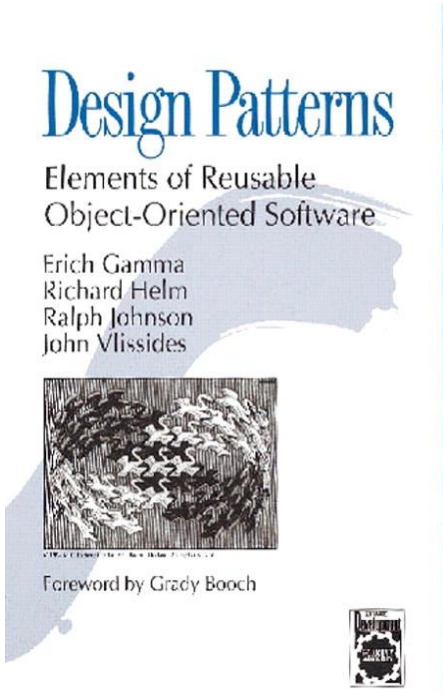




- **Frameworks MVC** (Trygve Reenskaugh), **ET++** (Erich Gamma), **JUnit** (Kent Beck y Gamma),…
- Artículos en **OOPSLA** (Beck y Cunnigham, 1987)
- **Hillside Group** (1993)
- Conferencias **PLoP** (1994)
- Libro **GoF** (1994)

Ingenieros en Informática al menos deberían conocer los patrones GOF (Recomendaciones ACM-IEEE)

"Reutilizar soluciones que funcionaron en el pasado"



- "Diseñar software orientado a objetos es difícil, pero diseñar software orientado a objetos **reutilizable** es más difícil todavía. Diseños generales y **flexibles** son muy difíciles de encontrar la primera vez"
- Programadores inexpertos podrían **aprovechar experiencia** de expertos a través de patrones.

"Una **arquitectura orientada a objetos bien estructurada está llena de patrones**. La calidad de un sistema orientado a objetos se mide por la atención que los diseñadores han prestado a las colaboraciones entre sus objetos."

"Los patrones **conducen a arquitecturas más pequeñas**, **simples** y **comprensibles**"

Grady Booch

Qué es un patrón de diseño

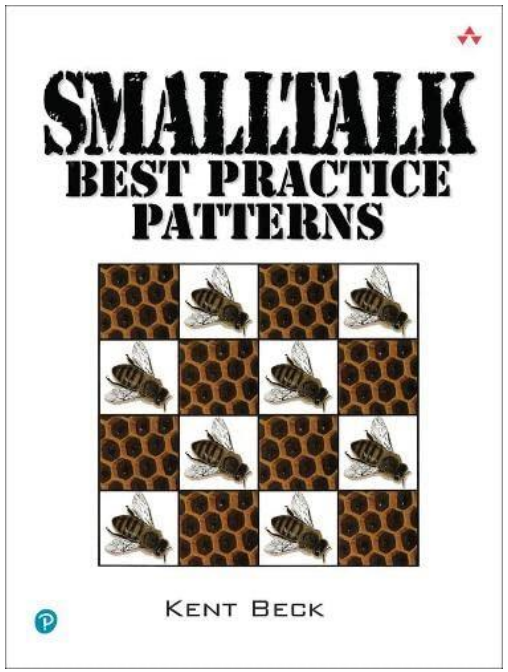
- Una solución reutilizable a un problema recurrente en la construcción de software y que funcionó en el pasado.
- *Lenguajes de patrones*
  - Cada patrón es identificado por un **nombre** y se describe mediante una **plantilla** cuyos elementos principales son:
    - Nombre
    - Problema
    - Solución: Descripción abstracta y reutilizable
    - Consecuencias

Hay otros tipos de “patrones software” además de “patrones de diseño”

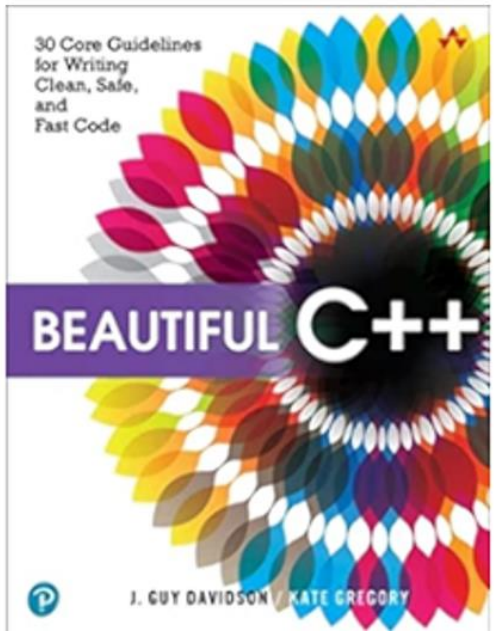
Tipos de patrones: Código

Patrones de Código

Buenas prácticas de programación y uso de las construcciones de un lenguaje de programación

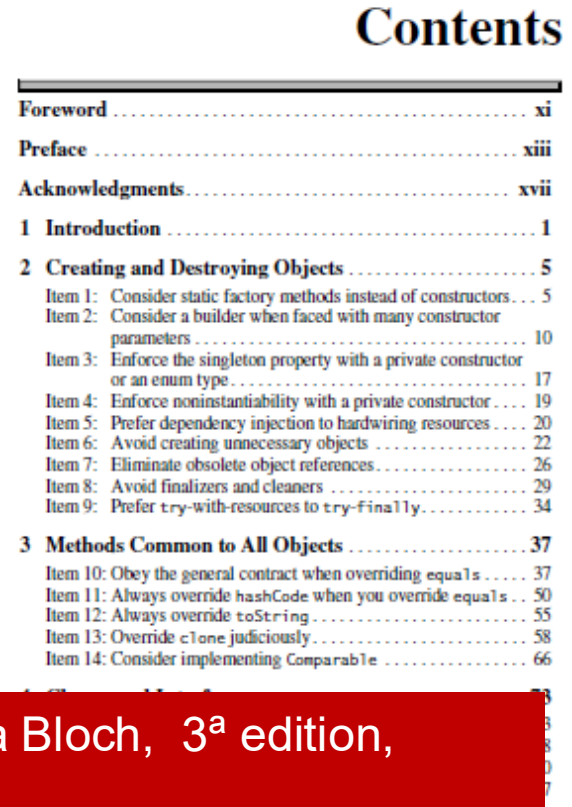


El primer libro de patrones de código



Después aparecieron para muchos lenguajes





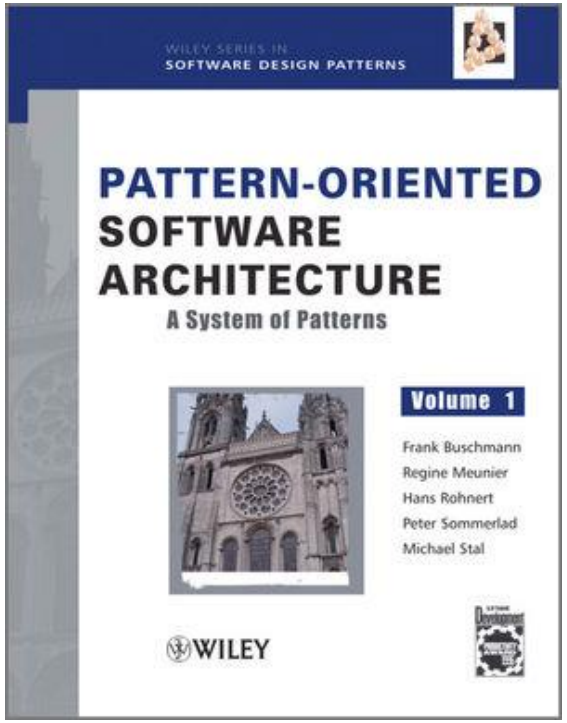
"Effective Java", Joshua Bloch, 3ª edition, Addison-Wesley, 2018

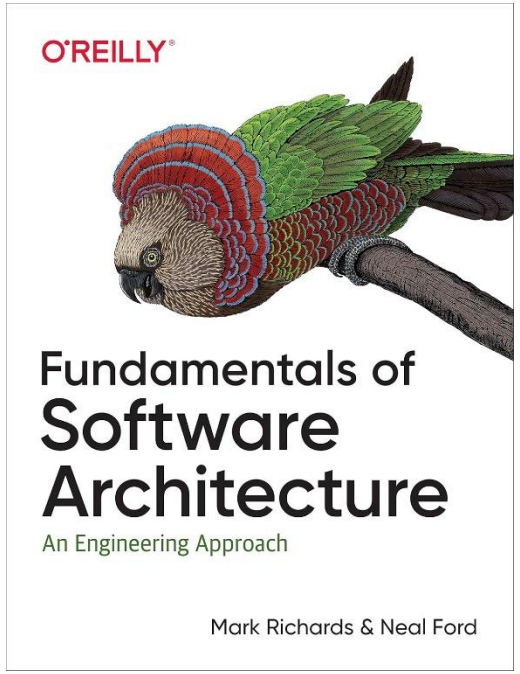
Excelente libro para desarrolladores Java

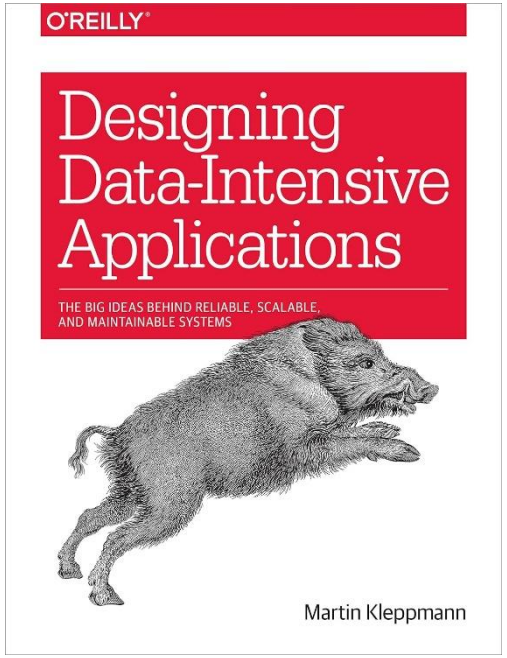
Tipos de patrones: Arquitectura

Patrones arquitecturales

Describen una organización del sistema. Ejemplos de arquitectura software: Layers, *Client-Server*, Microservicios, Lambda, o Event-Driven.

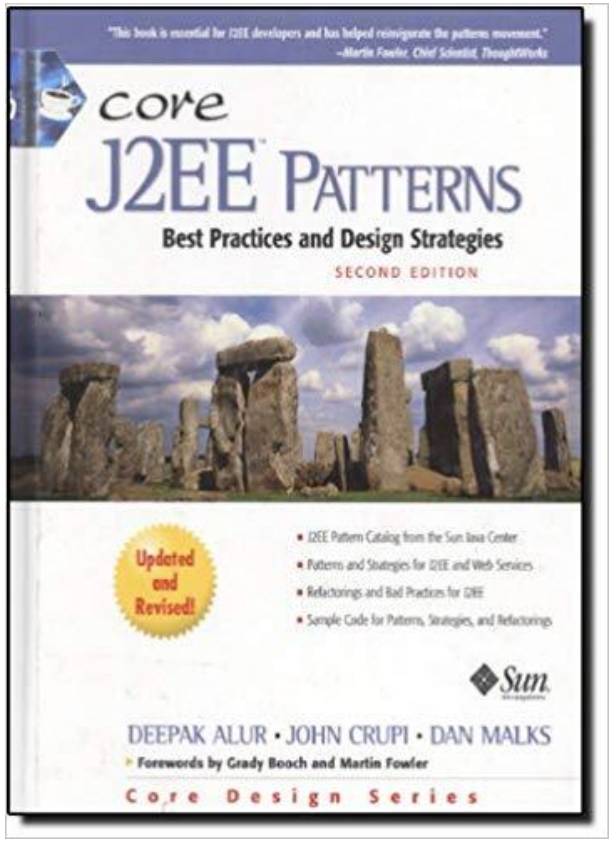


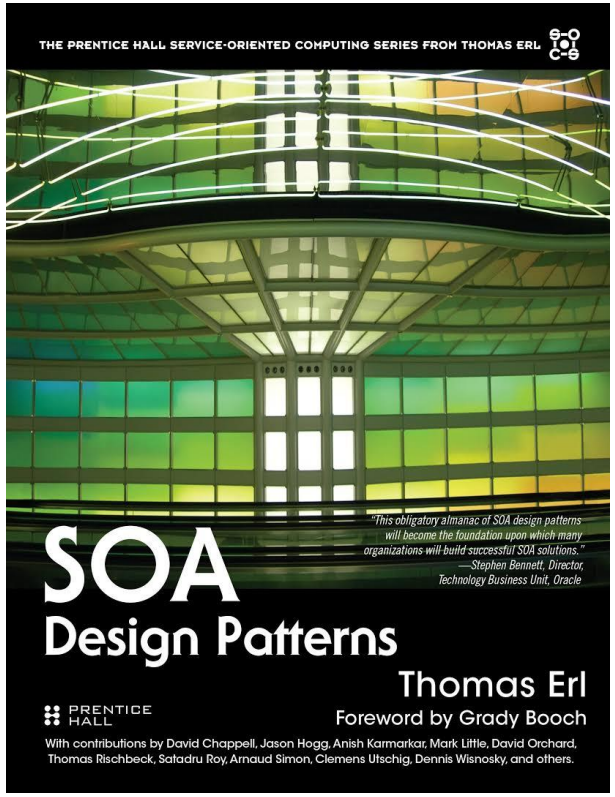


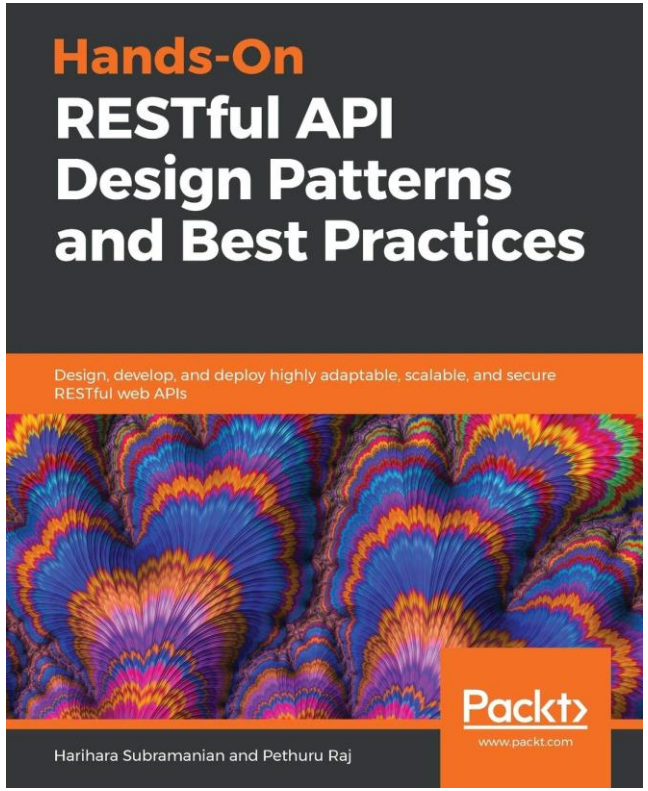


Tipos de patrones: Tecnologías específicas

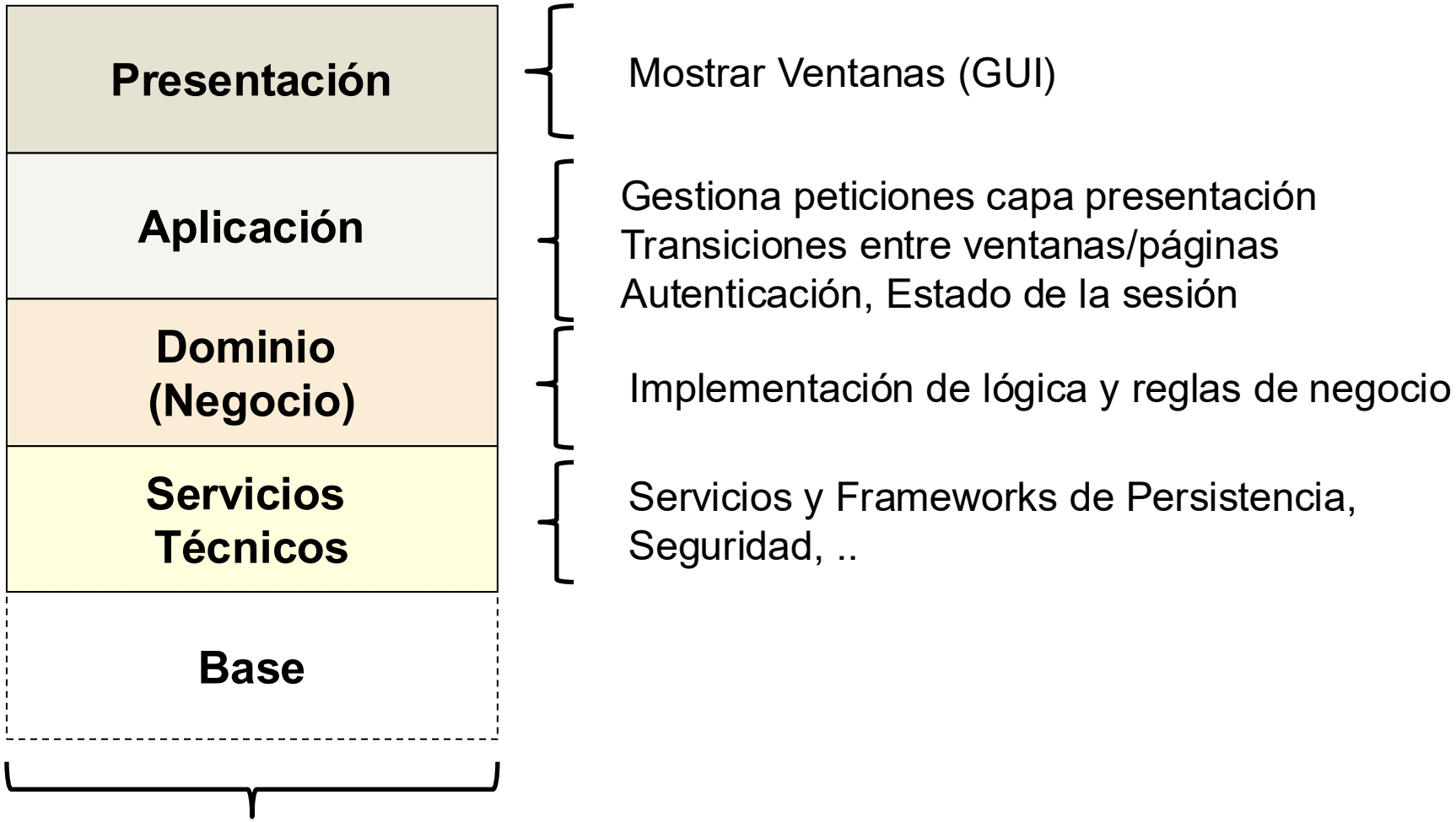
Patrones para tecnologías concretas: *J2EE, REST, SOA, ..*



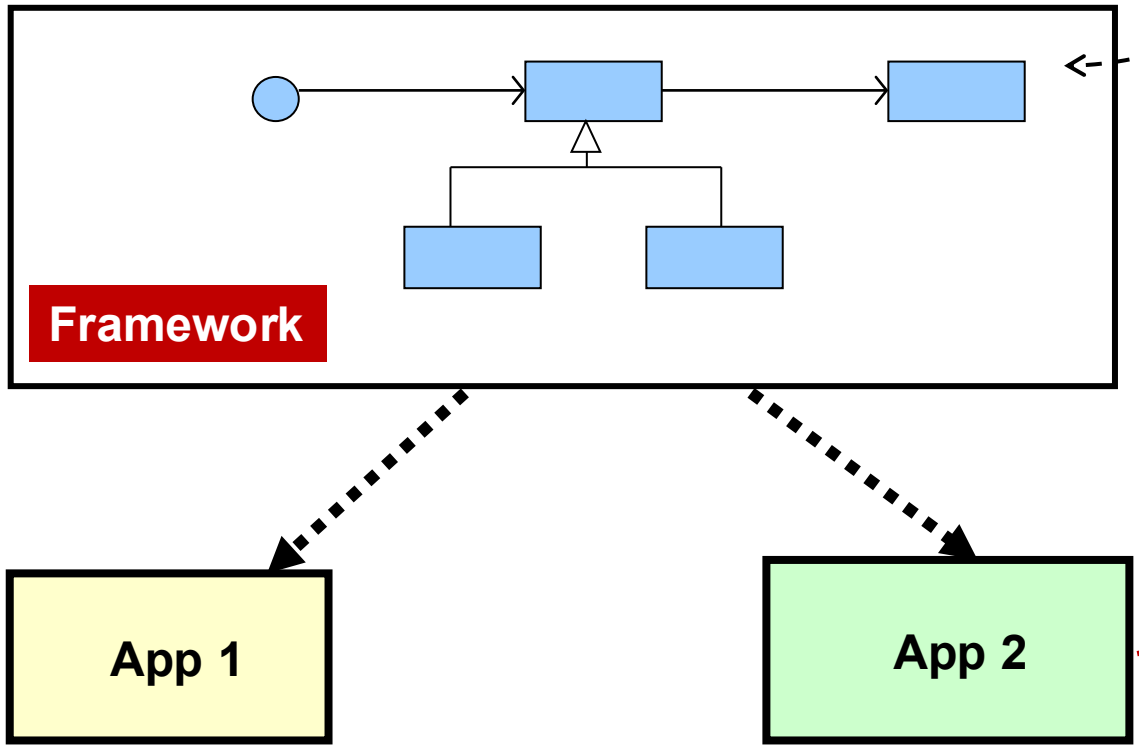




Patrón arquitectural “Layers”



Qué es un **framework**



Importante el uso de Patrones

Arquitectura reutilizable
para un dominio de aplicaciones

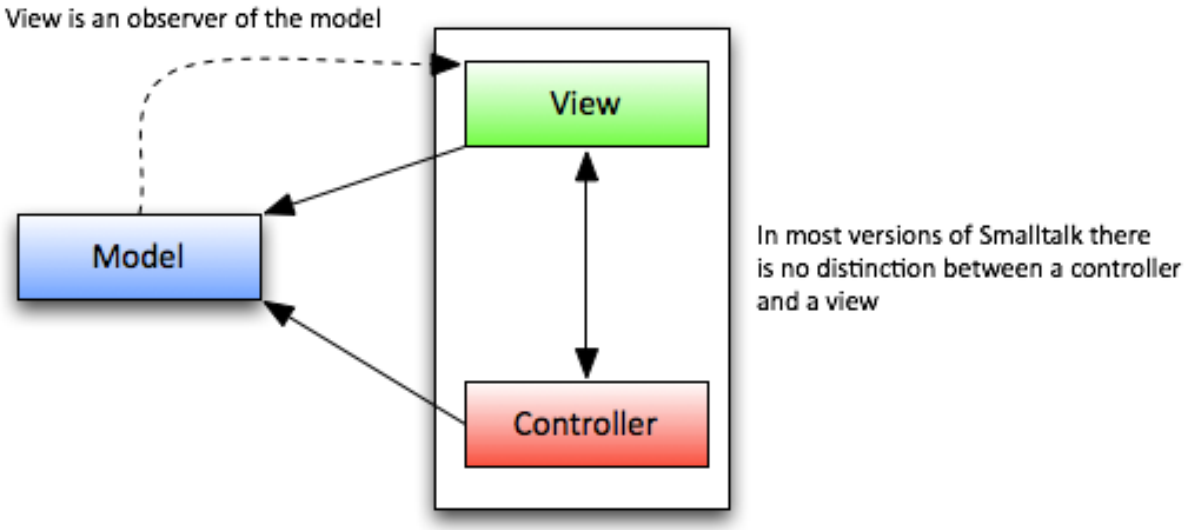
Aplicaciones concretas son creadas a partir del framewok

Qué es un framework

Un *framework* es una colección organizada de clases que constituyen un **diseño e implementación reutilizable para crear aplicaciones en un dominio**.

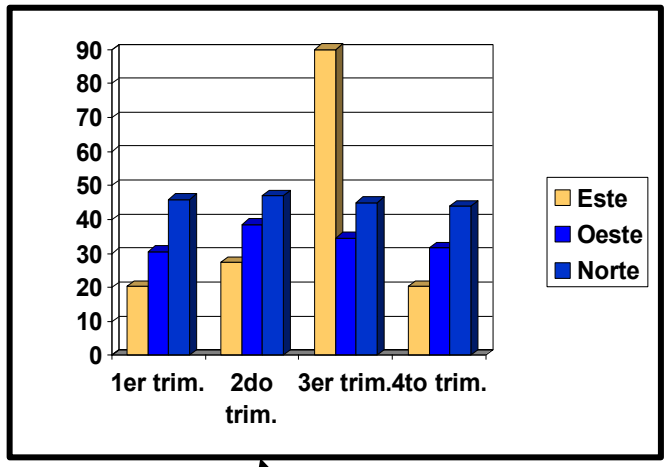
- Orientado a un dominio horizontal (aplicaciones web, seguridad, persistencia, editores gráficos, ..) o vertical (hospitales, finanzas, domótica, videojuegos ..)
- Establece la arquitectura de la aplicación (o parte).
- Necesario extenderlo para construir una aplicación concreta del dominio.
- Su construcción involucra muchos patrones de diseño.
- Ejemplos: Spring Boot, Angular, React, Node, JUnit, EMF/Eclipse, JPA/Hibernate, ET++, ..

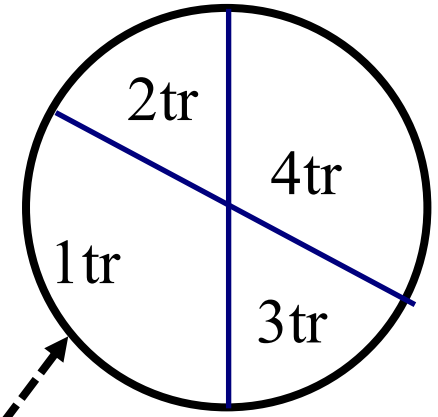
Framework Modelo-Vista-Control (MVC)



- Creado para construir aplicaciones interactivas gráficas en Smalltalk-80 **(Trygve Reenskaug).**
- Basado en tres tipos de objetos:
  - Modelo: objetos del dominio
  - Vista: objetos presentación en pantalla (interfaz de usuario)
  - Controlador: objetos que define cómo reacciona la interfaz y el modelo a la entrada del usuario.
- **Utiliza los patrones** Observer, Método Factoría, Estrategia, Decorador, Composite, entre otros.

*Patrón **Observer** permite desacoplar el modelo de la vista: un cambio en el modelo debe ser notificado a una o más vistas*





3 Vistas

	1tr.	2tr.	3tr.	4tr.
Este	20	25	90	20
Oeste	30	38	32	32
Norte	47	47	45	45

Modelo

t1=20
t2=25
t3=90
t4=20

Patrones de diseño. Plantilla de definición GoF

- **Nombre**
- **Propósito**

  ¿Qué problema aborda?
- **Sinónimos**
- **Motivación**

  Escenario que ilustra un problema particular y cómo el patrón lo resuelve.
- **Aplicabilidad**

  ¿En qué situaciones puede aplicarse?
- **Estructura**

  Diagrama de clases que ilustra la estructura
- **Participantes**

  Clases que participan y sus responsabilidades
- **Colaboraciones**

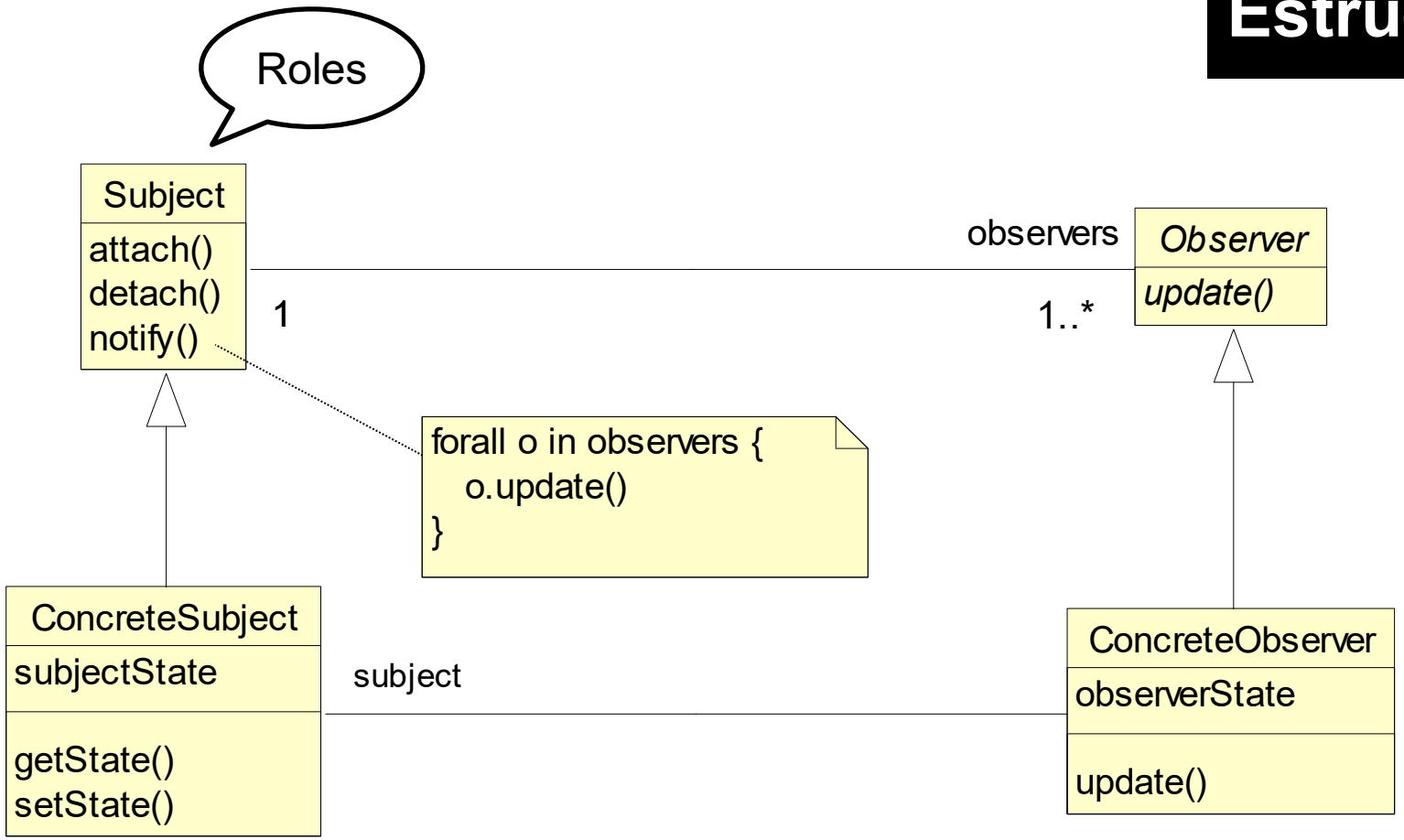
  Diagramas de interacción que muestran cómo colaboran los participantes.
- **Consecuencias**

  ¿Cómo alcanza el patrón sus objetivos?, ¿Cuáles son los compromisos y resultados de usar el patrón?, Alternativas, Costes y Beneficios.
- **Implementación**

  Técnicas, heurísticas y consejos para la implementación ¿Hay cuestiones dependientes del lenguaje?
- **Ejemplo de Código**
- **Usos conocidos**
- **Patrones relacionados**

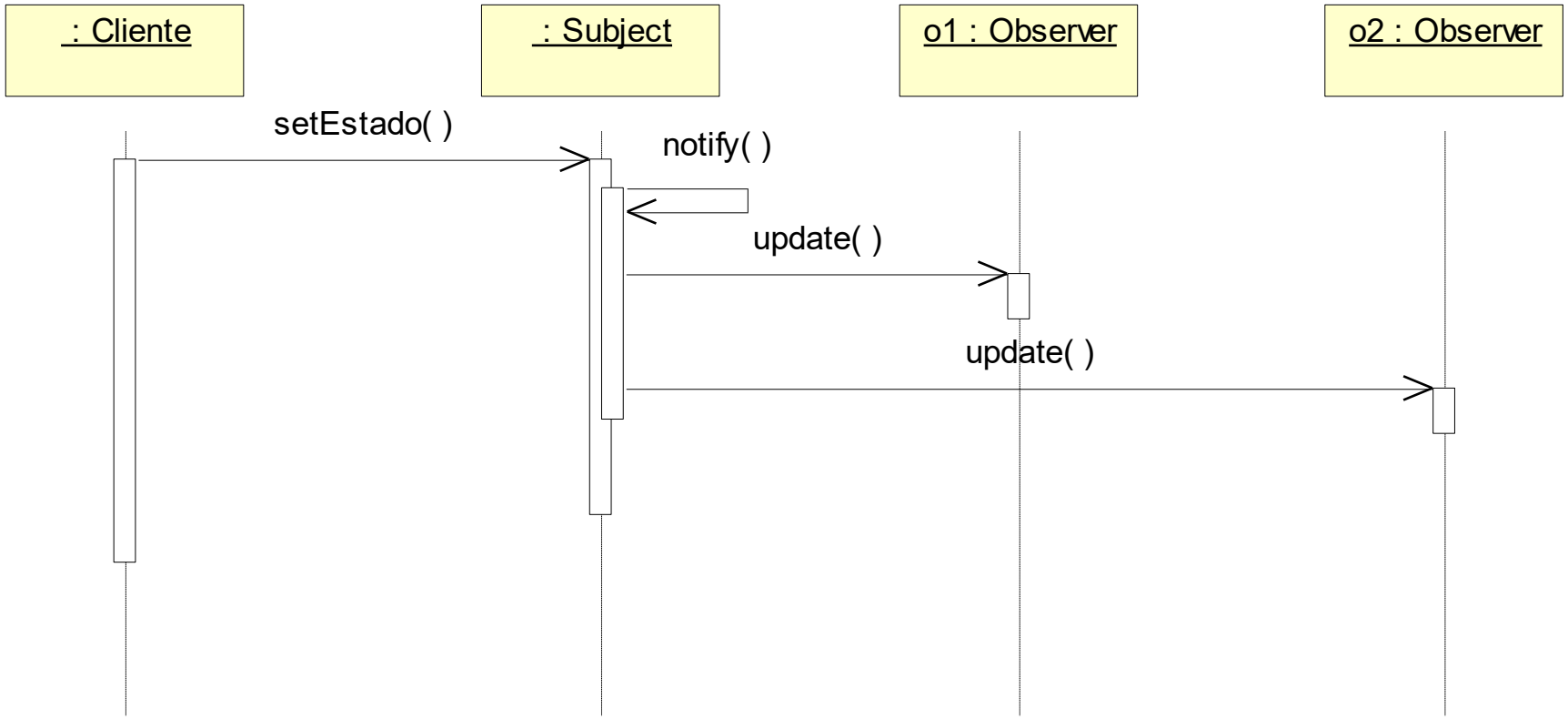
Patrón
Observer

Estructura



Patron **Observer**

Comportamiento



Clasificación

		Propósito		
		Creación	*Estructural*	*Comportamiento*
Solución	Herencia	**Factory Method**	Adapter	**Template Method**
	Composición	**Abstract Factory** **Builder** **Factory Method** Prototype **Singleton**	**Adapter** **Bridge** **Composite** **Decorator** **Facade** Flyweight **Proxy**	Chain of Responsability **Command** **Iterator** Mediator **Memento** **Observer** **State** **Strategy** Visitor

¿A qué ayudan los patrones?

- Principalmente favorecen la **extensibilidad:** *Asociar patrones a puntos de variación*
- En mucha menor medida, favorecen la **reutilización**
- Facilitan comunicación, documentación y comprensión

¿Cómo ayudan?

Aplican dos principios básicos de programación OO

- Programar hacia la interfaz, no hacia la implementación
- Favorecer la composición en vez de la herencia

Causas comunes de rediseño debido a dependencias

i) **Crear un objeto indicando la clase** - - > Factorías

ii) Dependencia de **operaciones específicas** - - > Command

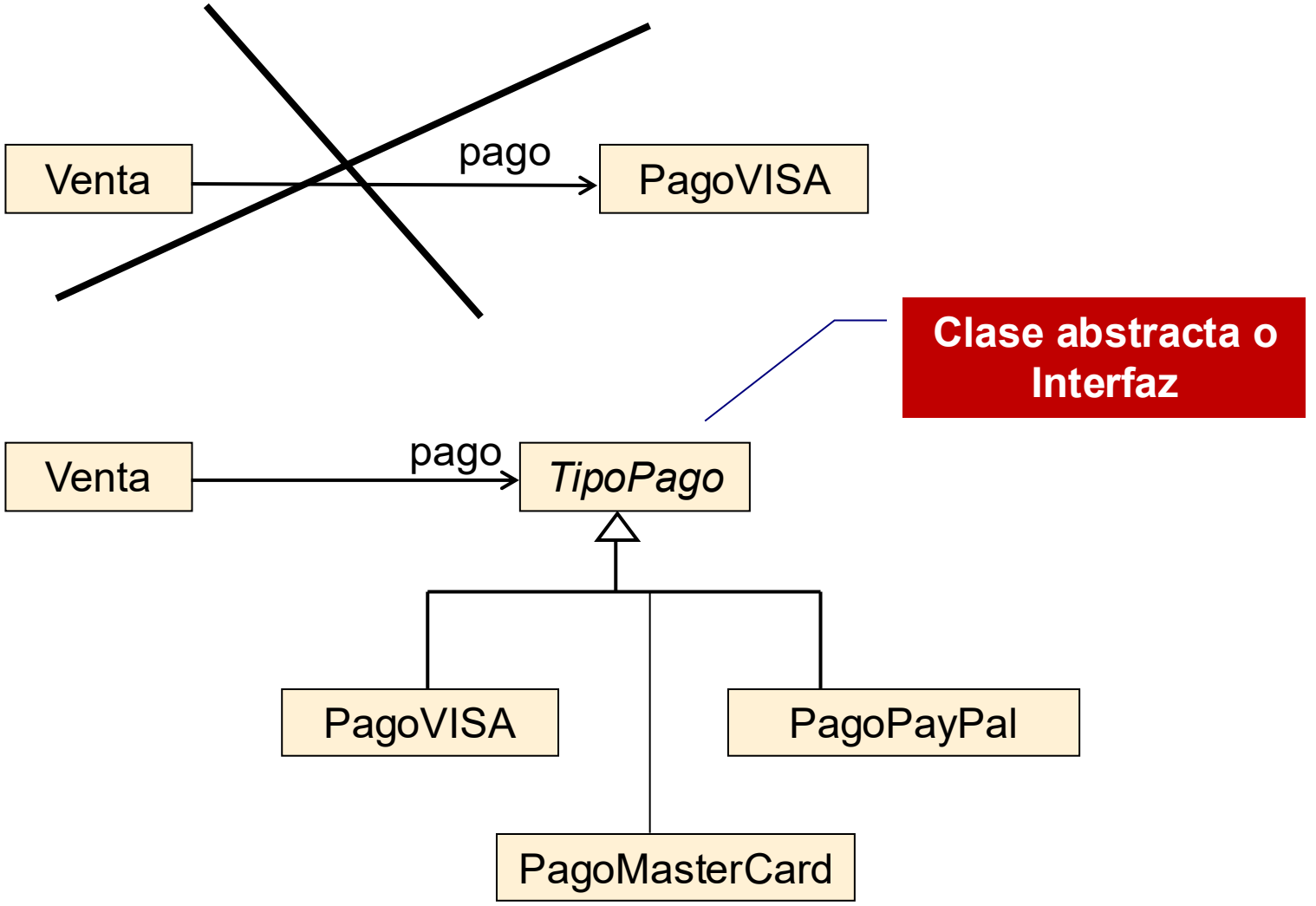
iii) Dependencia de **plataformas** hardware o software - - > Factoría Abstracta y Bridge

iv) Dependencia sobre **representación de objetos** - - > Factoría Abstracta y Bridge

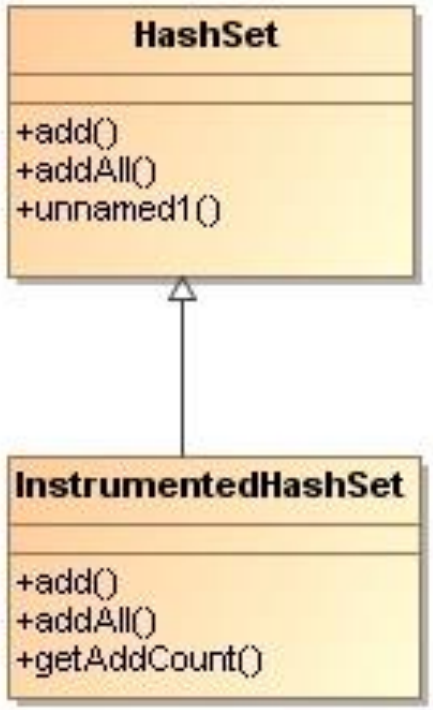
v) Dependencias de **algoritmos** - - > Estrategia

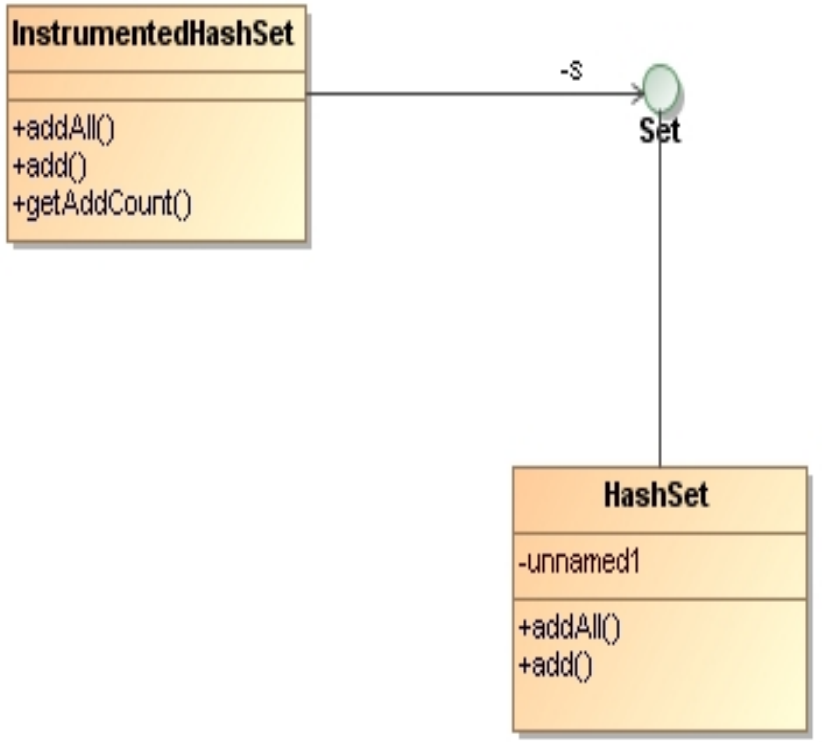
vi) **Acoplamiento** fuerte entre clases - - > Fachada, Observer

Programar hacia la interfaz



Favorecer composición





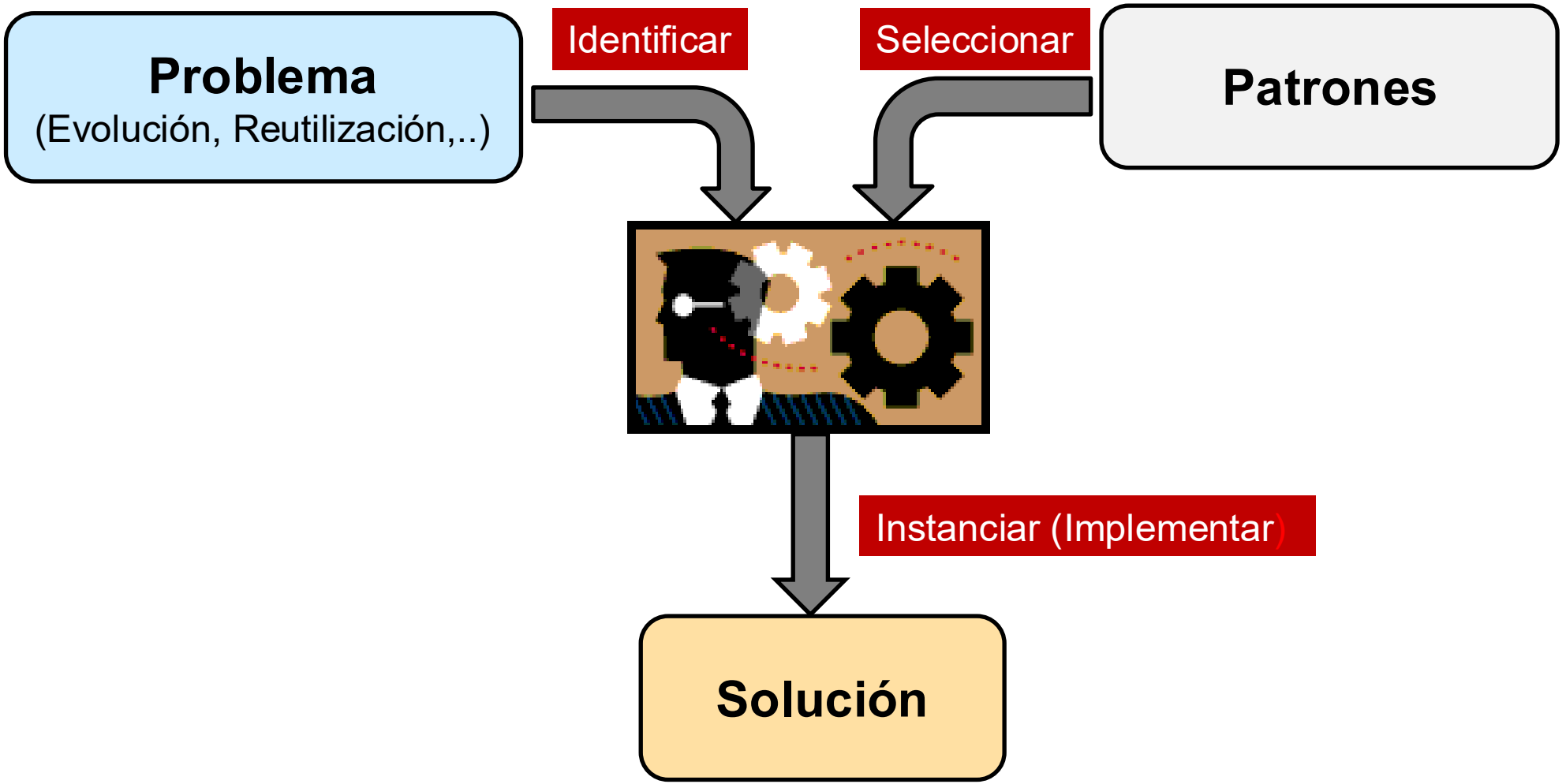
Relación **fija**

Reutilización **"caja blanca"**

Relación **variable**

Reutilización **"caja negra"**

¿Cómo seleccionar y aplicar un patrón?

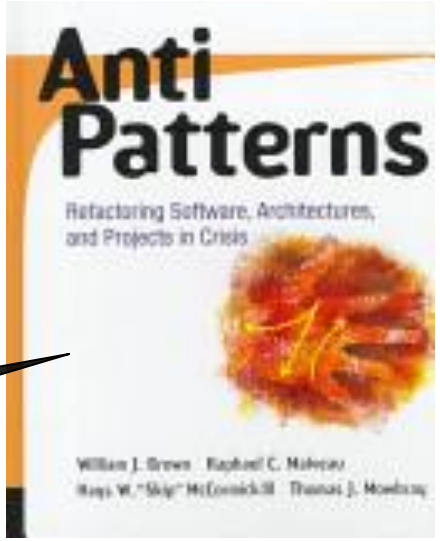


Patrones vs. Antipatrones y Refactoring

Un anti-patrón es una mala práctica en el desarrollo de software (análisis, diseño, e implementación).

- **Ejemplo**: *clase Dios*

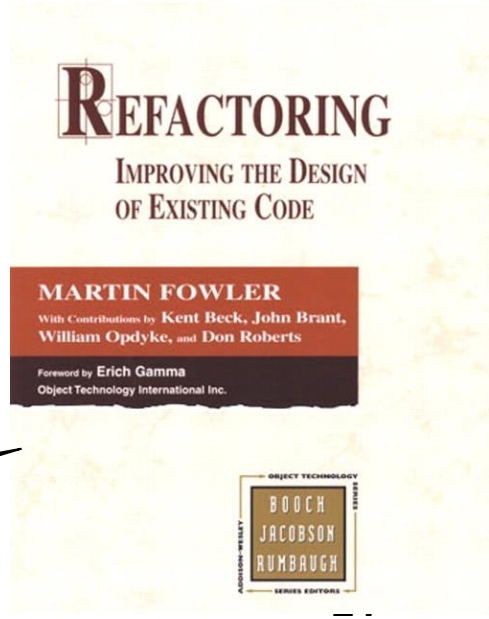
¡Ningún Éxito!



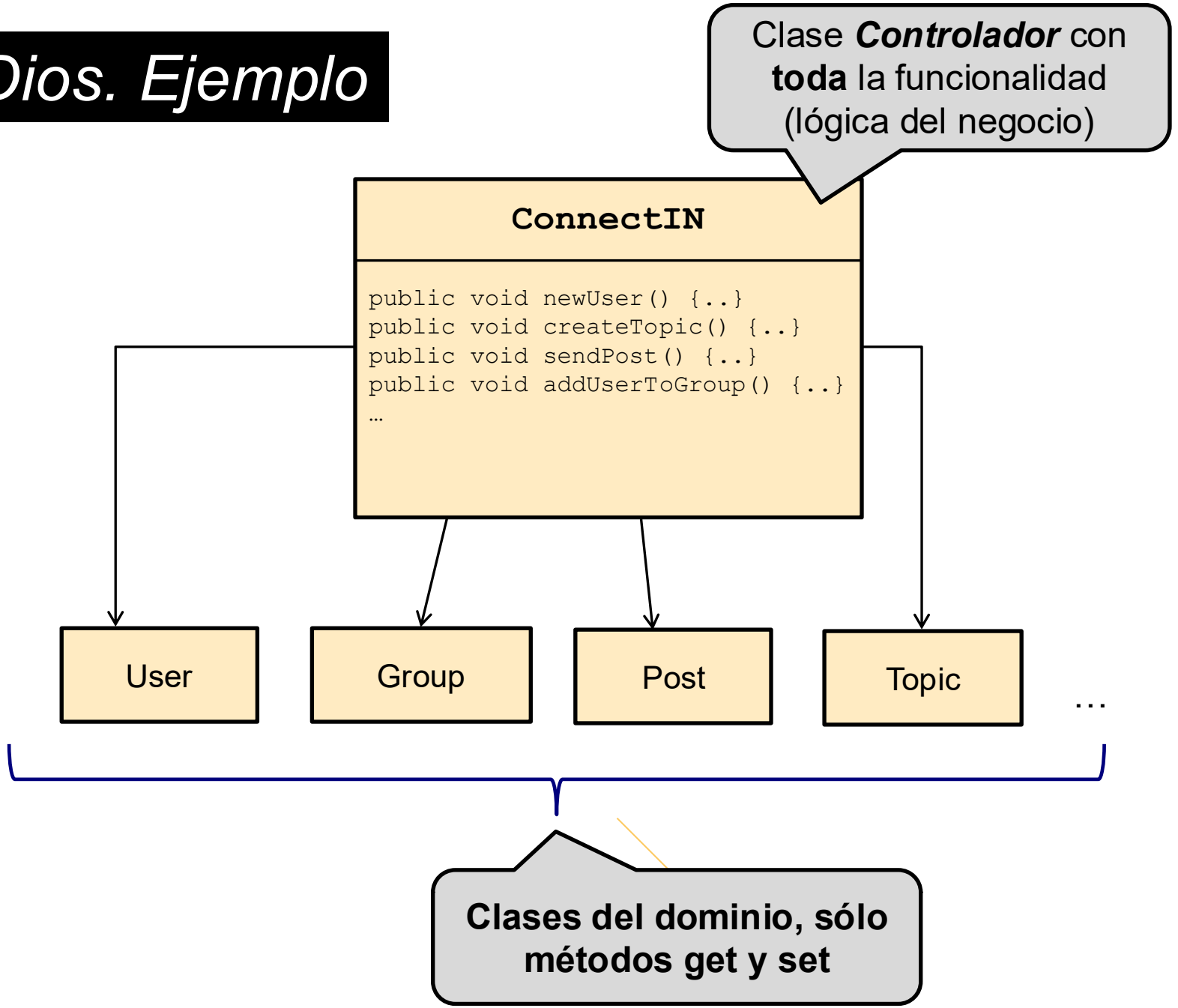
Un refactoring es un cambio en la estructura interna de un programa para mejorar su calidad (comprensión, evolución,..) sin cambiar su comportamiento.

- Ejemplos: código duplicado, clase grande, expresión condicional compleja, eliminar jerarquía, …

¡Gran Éxito!



Clase *Dios. Ejemplo*



Patron-itis: Abuso del uso de patrones

← Singleton in multi-threaded environment

The relation between Integer Partition and Pentagonal Numbers →

The Abuse of Design Patterns in writing a Hello World Program

Posted on September 21, 2011

"One comment I saw in a news group just after patterns started to become more popular was someone claiming that in a particular program they tried to use all 23 GoF patterns. They said they had failed, because they were only able to use 20. They hoped the client would call them again to come back again so maybe they could squeeze in the other 3.

Trying to use all the patterns is a bad thing, because you will end up with synthetic designs—speculative designs that have flexibility that no one needs. These days software is too complex. We can't afford to speculate what else it should do. We need to really

StackOverflow



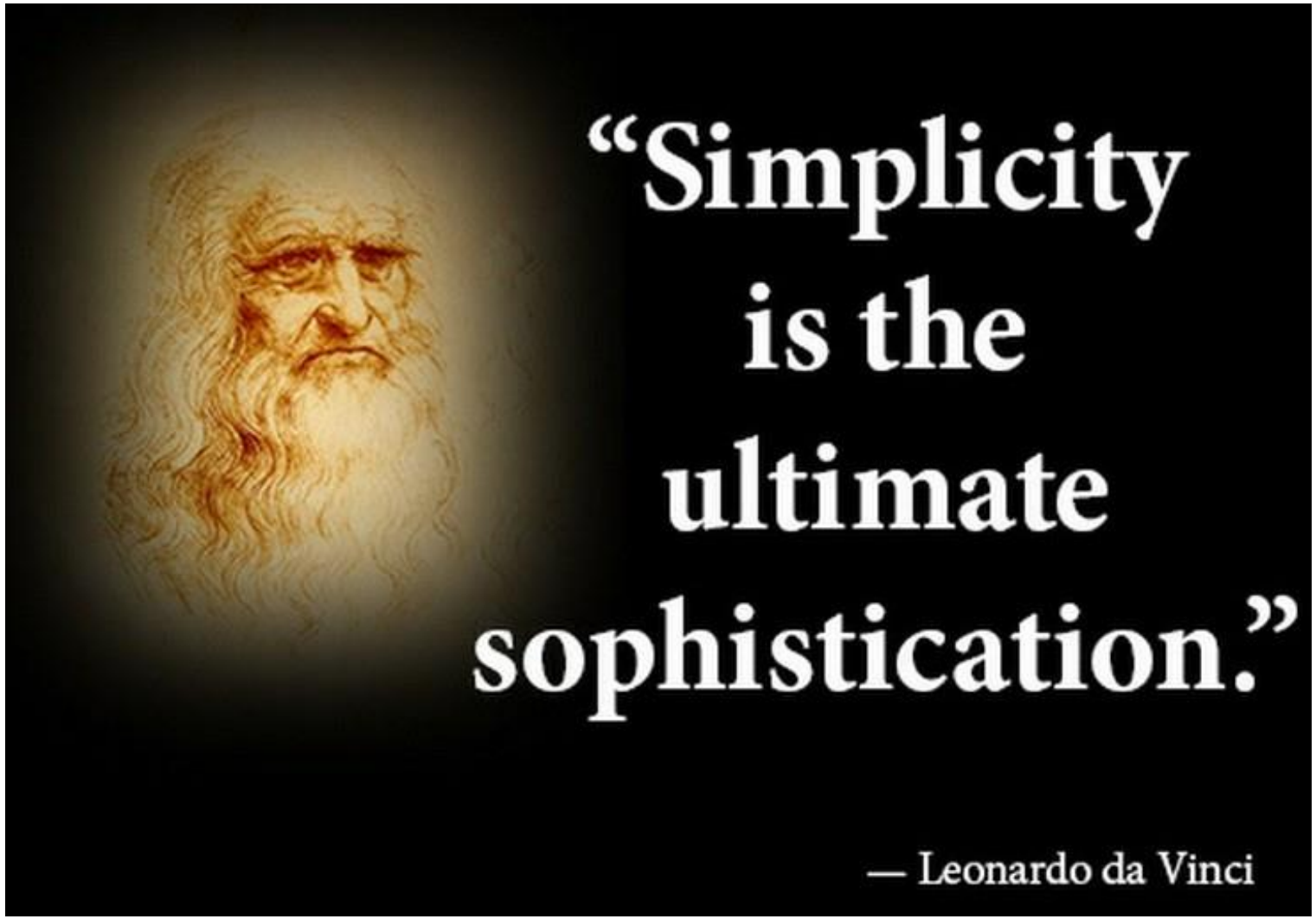
taskinoor
42,964
●10 ●109 ●135

Categories

- Algorithm (1)
- AS3 (3)
- Design Pattern (4)
- Linux (2)

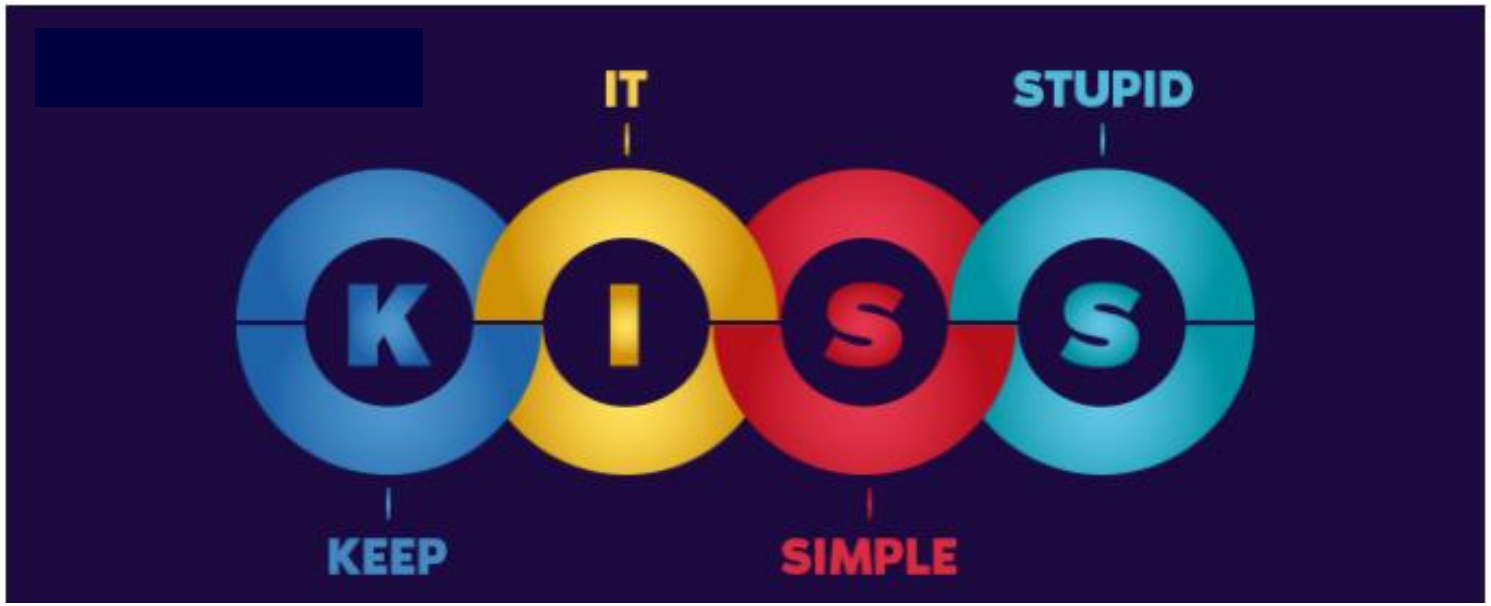
Archives

- November 2011 (1)
- September 2011 (1)
- April 2011 (2)
- February 2011 (3)
- January 2011 (2)


“Simplicity is the ultimate sophistication.”
— Leonardo da Vinci

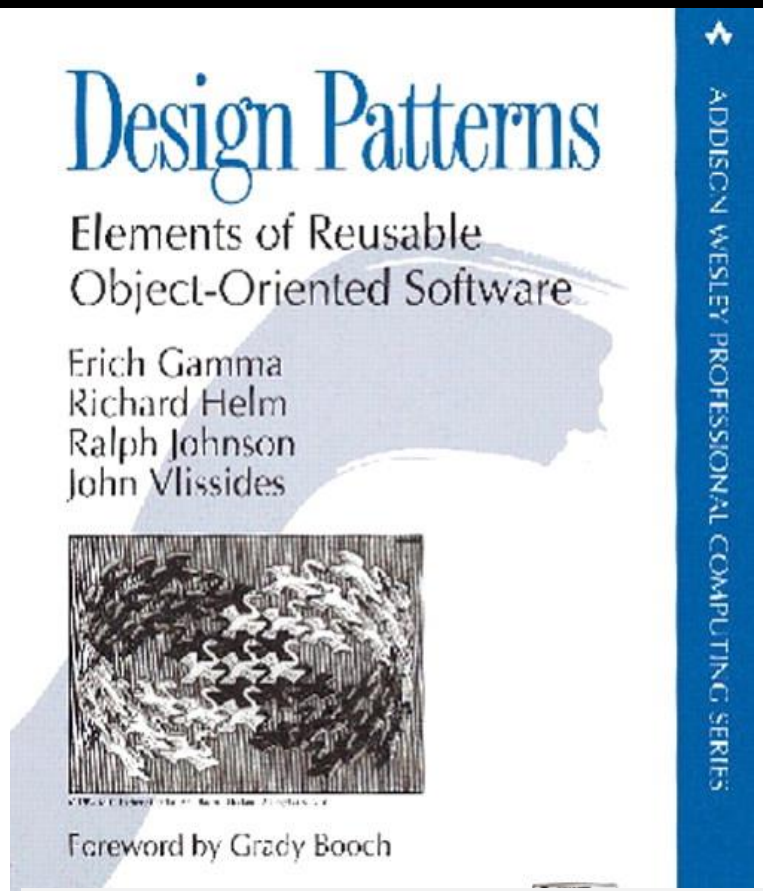
The KISS Principle in Software Development — Everything You Need to Know

Unthinkable Software Nov 20, 2019 · 5 min read



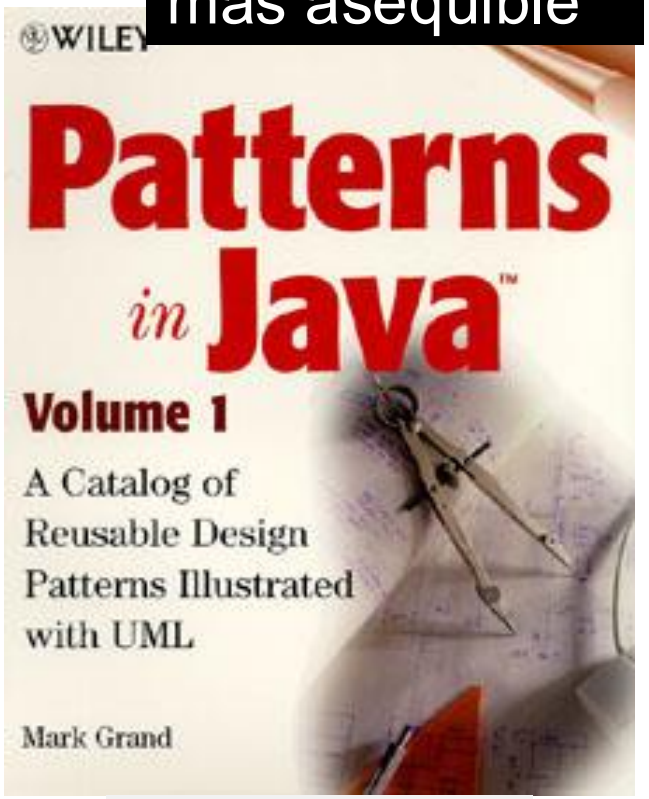
https://medium.com/sliitwif/keep-it-simple-stupid-the-kiss-principle-guide-to-developers-d6ad83145955

Obra “seminal”, ¿lectura recomendable hoy?



Design Patterns,
E. Gamma et al.
Addison-Wesley, 1995
(Versión española de 2003)

Lectura
más asequible

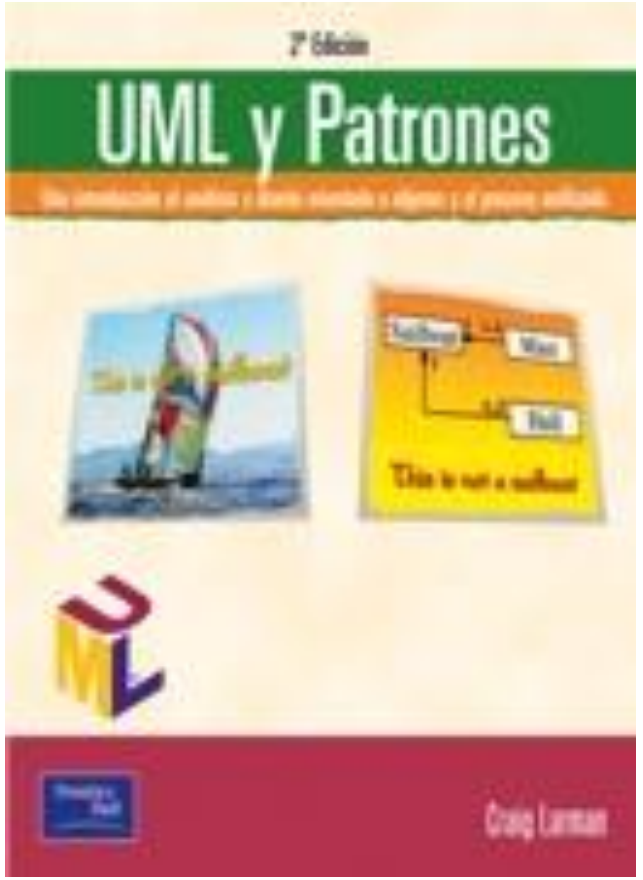


Patterns in Java,
Mark Grand
vol. 1, 2ª edición,
John Wiley, **1998**

Éxito entre los alumnos, también para Java



Explica los patrones básicos e ilustra su uso para el caso práctico del TPV/Tienda



UML y Patrones. Introducción al análisis y diseño orientado a objetos, Craig Larman Prentice-Hall, 2ª edición, **2002**

www.vincehuston.org/dp/

Dozens of 1-page examples in C++ and Java. Before-and-After refactoring examples. Book summaries.

Design Patterns

http://www.vincehuston.org/dp

Vince Huston



new software design blog



The Sacred Elements of the Faith

the holy origins		the holy behaviors			the holy structures		
							139 A Adapter
107 FM Factory Method							
117 PT Prototype	127 S Singleton				223 CR Chain of Responsibility	163 CP Composite	175 D Decorator
87 AF Abstract Factory	325 TM Template Method	233 CD Command	273 MD Mediator	293 O Observer	243 IN Interpreter	207 PX Proxy	185 FA Façade
97 BU Builder	315 SR Strategy	283 MM Memento	305 ST State	257 IT Iterator	331 V Visitor	195 FL Flyweight	151 BR Bridge

Design Pattern: identify a popular design strategy and document the domain-independent leverage.
Adapter: use an extra level of indirection to make 2 incompatible objects compatible.



[GoF Design Patterns]

Python Design Patterns

Welcome!

I'm **Brandon Rhodes** (website, Twitter) and this is my evolving guide to design patterns in the Python programming language.

- This site is letting me collect my ideas about Python and Design Patterns all in one place.
- My hope is that these pages make the patterns more discoverable — easier to find in web searches, and easier to read — than when they were scattered across the videos and slides of my Python conference talks.
- The weight of other obligations makes my progress intermittent. To check for new material, simply visit the commit history of this site's project repository on GitHub, where you can also select "Watch" to get updates.

With those preliminaries complete, here are the patterns!

Gang of Four: Principles

- The Composition Over Inheritance Principle
    - Problem: the subclass explosion

software "design pattern"

All Images Videos News Maps More Settings Tools

About 6,700,000 results (0.76 seconds)

en.wikipedia.org › wiki › Software_design_pattern

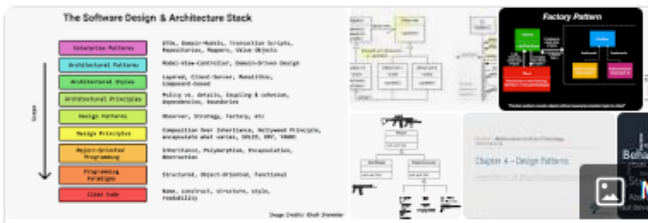
Software design pattern - Wikipedia

In software engineering, a **software design pattern** is a general, reusable solution to a commonly occurring problem within a given context in software design.

Pattern-Oriented Software ... · Category:Software design ... · Builder pattern · Servant

People also ask

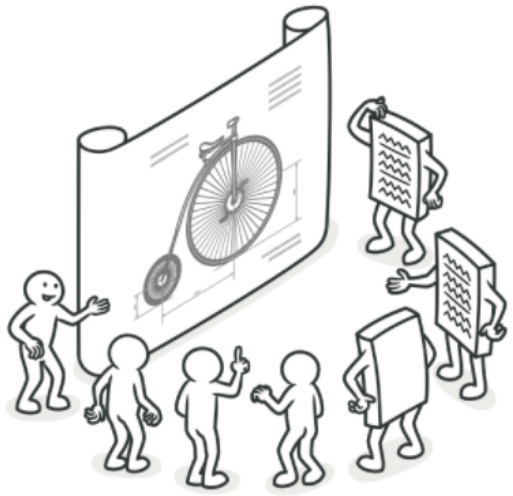
- How many software design patterns are there?
- What are the types of design patterns?
- What are the 23 design patterns?



Software design pattern

In software engineering, a software design pattern is a ger reusable solution to a commonly occurring problem within context in software design. It is not a finished design that c transformed directly into source or machine code. Wikiped

refactoring.guru/design-patterns/



DESIGN PATTERNS

Design patterns are typical solutions to common problems in software design. Each pattern is like a blueprint that you can customize to solve a particular design problem in your code.

What's a design pattern?



Benefits of patterns

Patterns are a toolkit of solutions to common problems in software design. They define

Classification

Design patterns differ by their complexity, level of detail and scale of applicability. In addition,

Quora Home 1 Answer Spaces Notifications 3 Search Qu

Software Engineering Books +6

What are the best books for design patterns?

Answer Follow · 76 Request

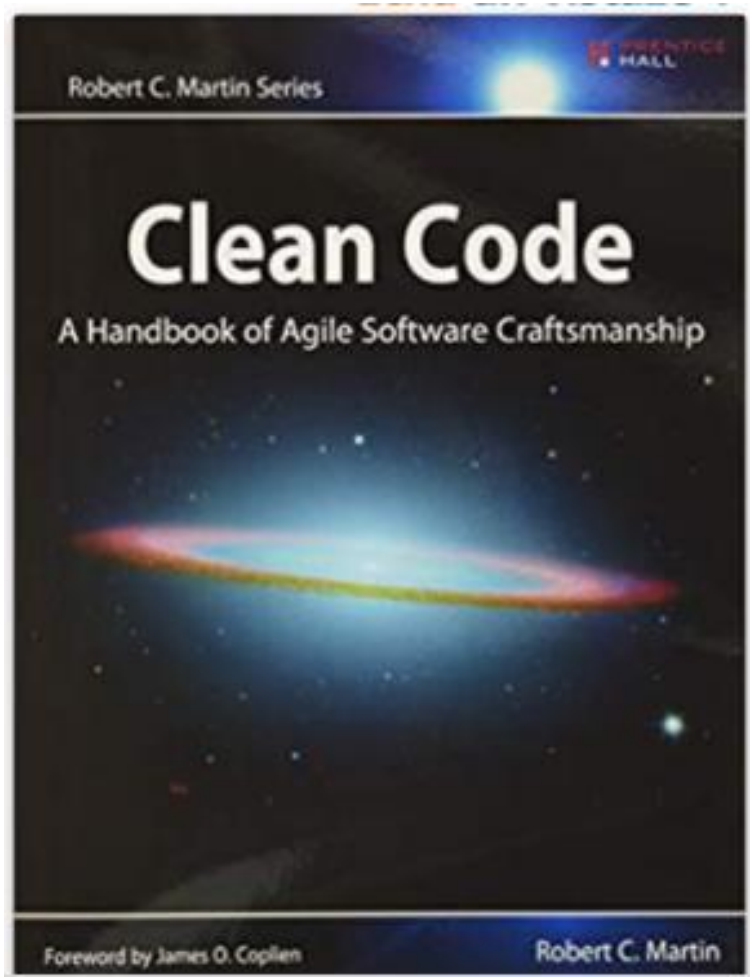
9 Answers

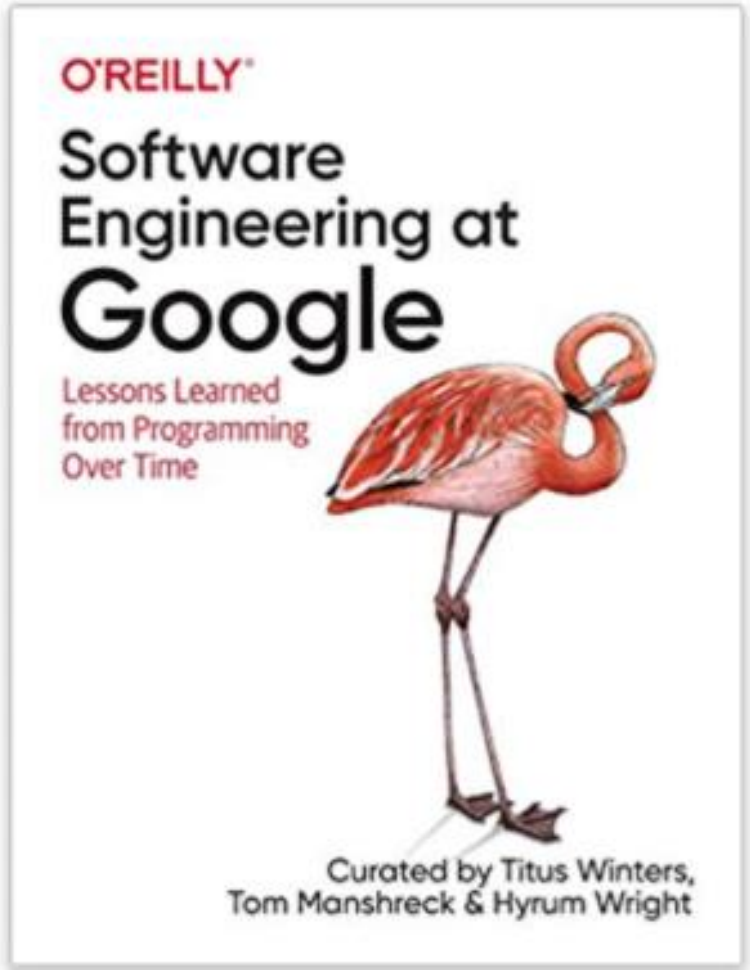


Kartik Sura, Engineer at a Startup
Answered Nov 2, 2013

My personal opinion is that Gang of Four is good for academic purposes (only), or as a theoretical reference.

Lecturas interesantes para un ingeniero software

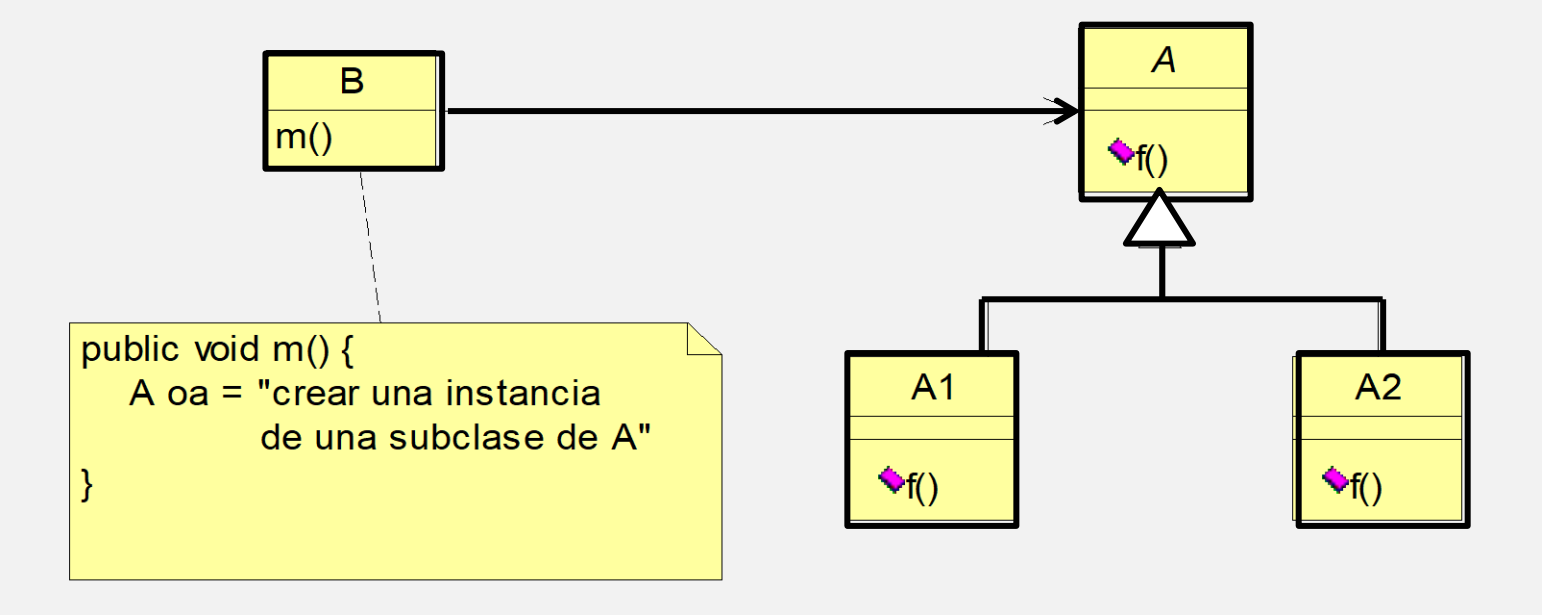




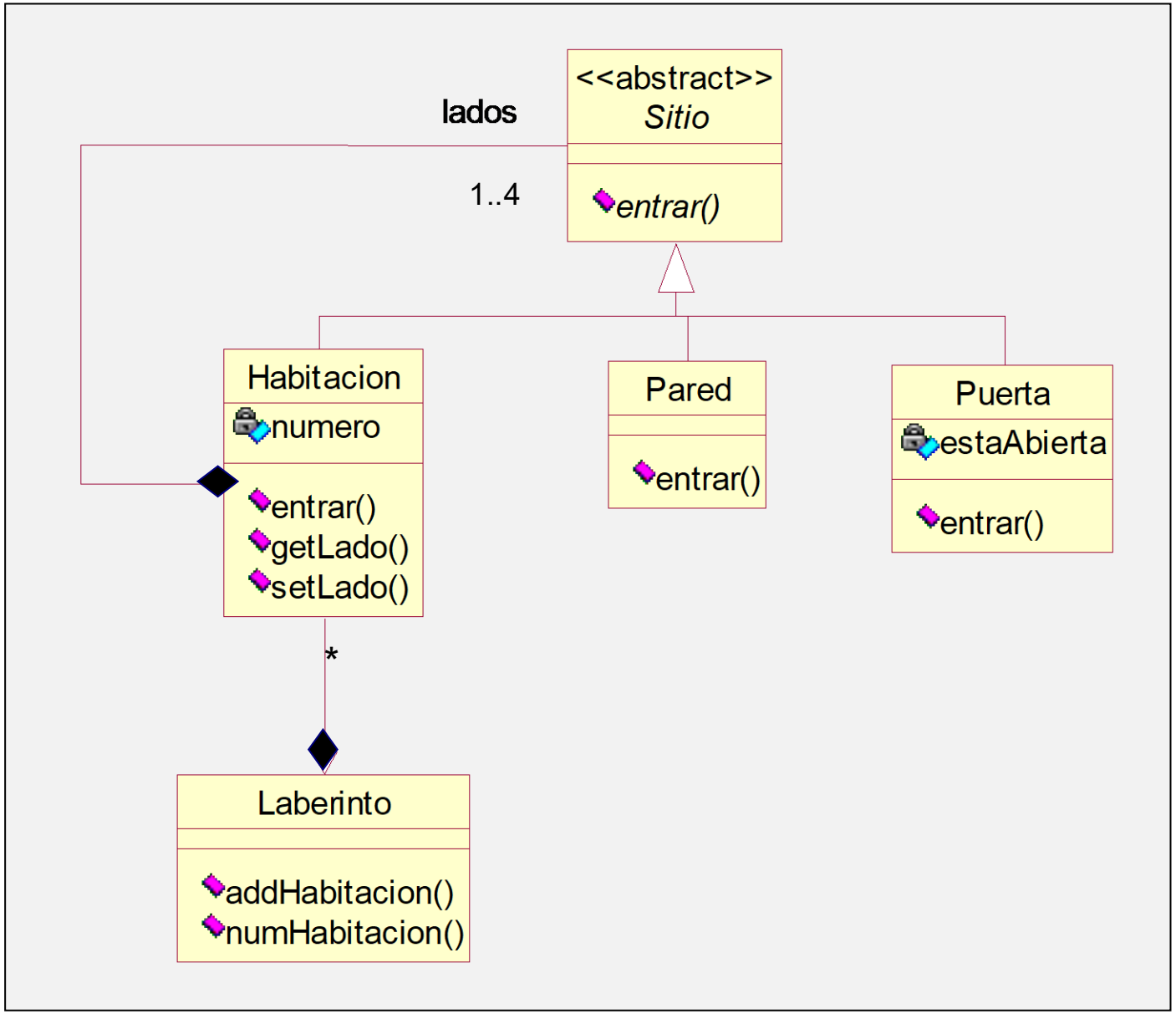
Patrones de Creación

Patrones de Creación

- Ayudan a crear aplicaciones independientes de cómo los objetos son creados, compuestos y representados.
- Se conoce una clase abstracta o interfaz, **pero no** se conoce la clase concreta a instanciar.



Ejemplo: Creación de un objeto `Laberinto`



```
public class JuegoLaberinto {..
   public Laberinto construirLaberinto () {
        Laberinto lab = new Laberinto();
        Habitacion h1 = new Habitacion(1);
        Habitacion h2 = new Habitacion(2);
        Puerta unaPuerta = new Puerta(1,2)

        lab.addHabitacion (h1);
        lab.addHabitacion (h2);

        h1.setLado(Norte, new Pared() );
        h1.setLado(Sur, new Pared() );
        h1.setLado(Este, new Pared() );
        h1.setLado(Oeste,unaPuerta);

        h2.setLado(Norte, new Pared);
        …
        return lab;
     }
}
```

Dependencia de clases concretas

Estructura del laberinto es fija

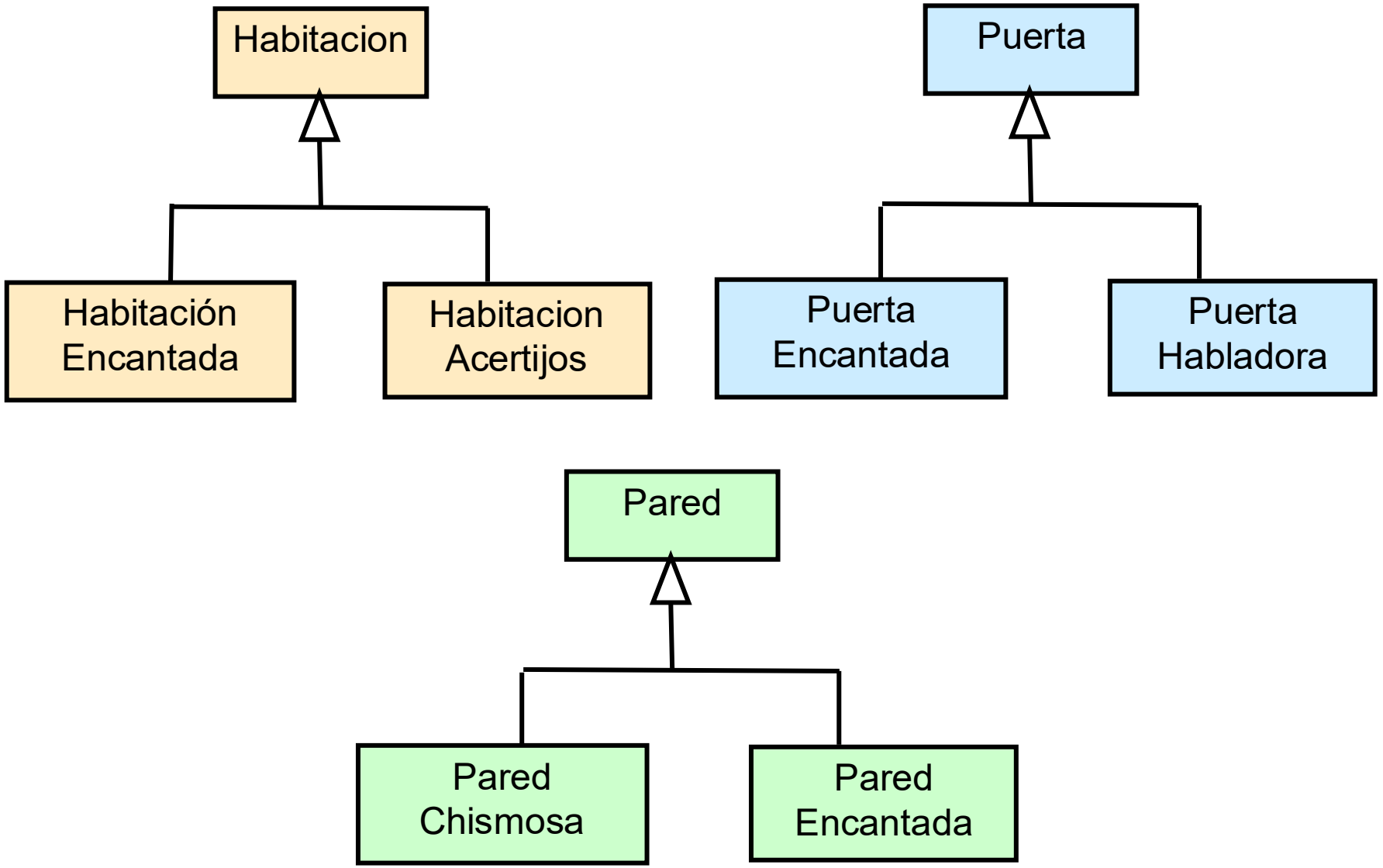
Ejemplo: Creación de un objeto `Laberinto`

Poco flexible:

¿Qué implicaciones tiene crear laberintos con otros tipos de elementos como *habitacionesEncantadas* o *puertasQueEscuchan*?

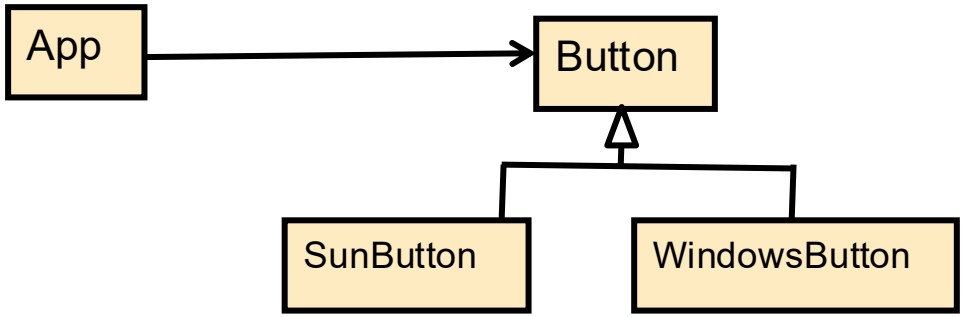
Patrones de creación permiten **eliminar referencias explícitas a clases concretas** desde el código que necesita crear instancias de esas clases.

Ejemplo: Creación de un objeto `Laberinto`

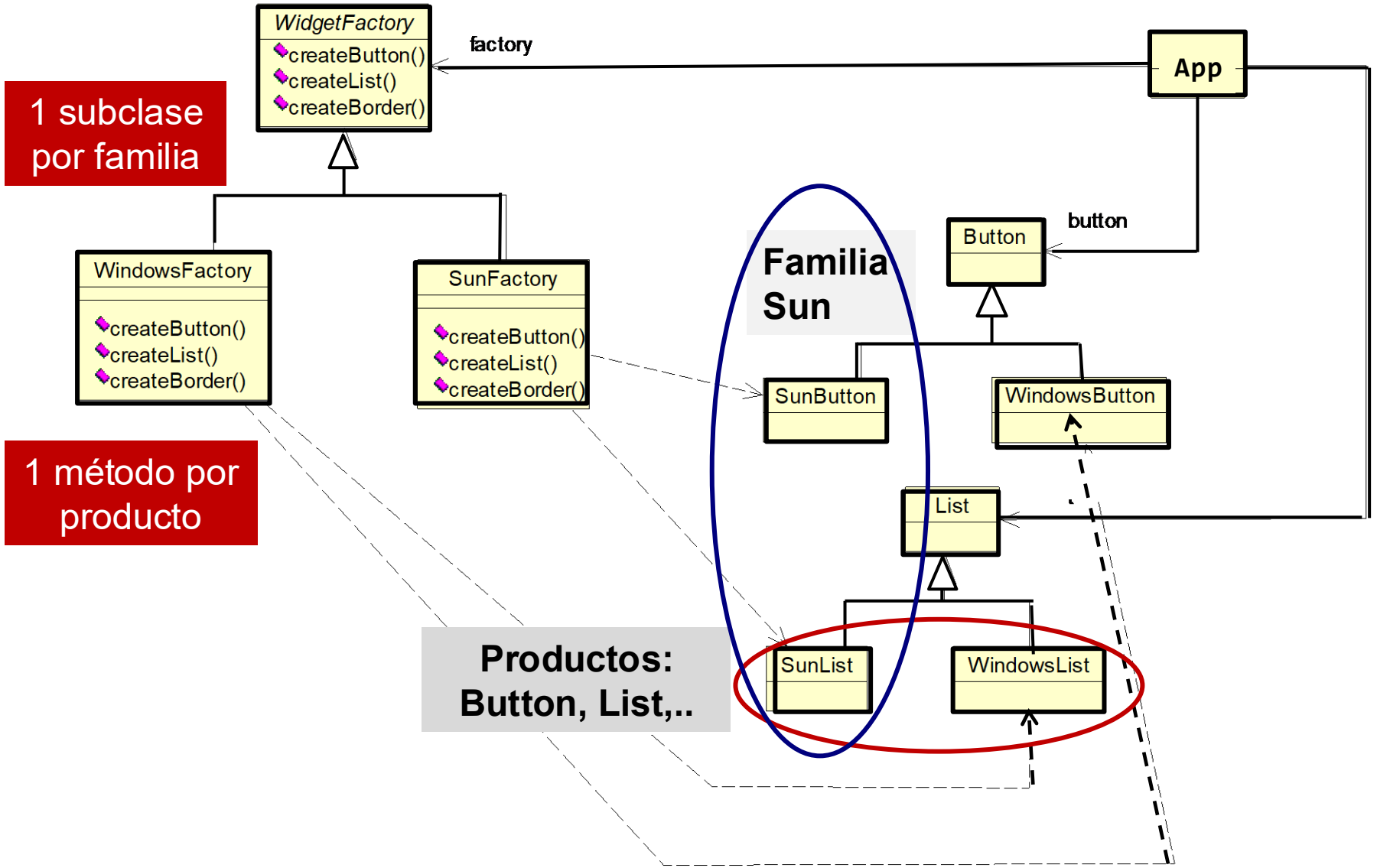


Abstract Factory (Factoría Abstracta)

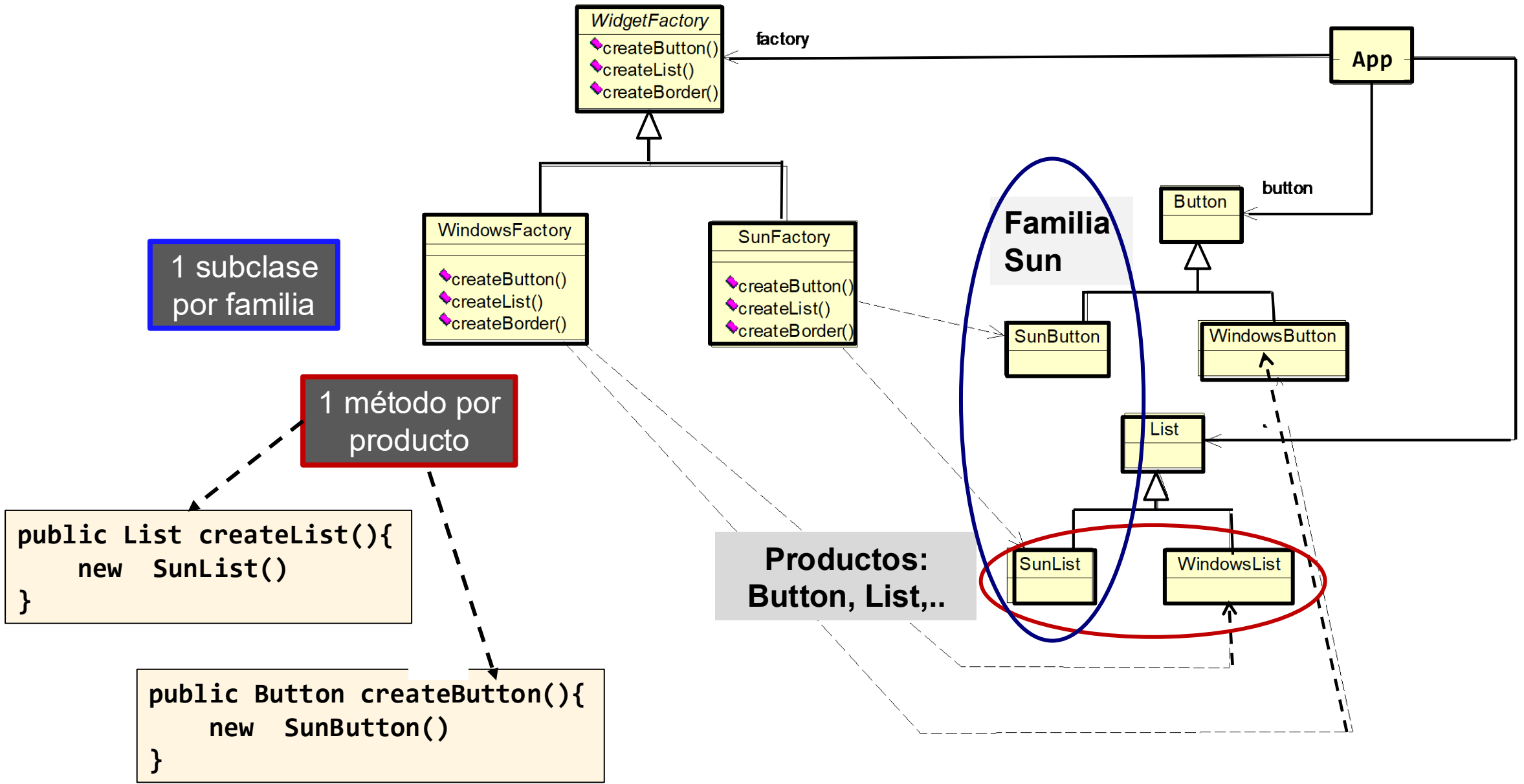
- **Propósito**
  - Proporcionar una interfaz para crear **familias de objetos** relacionados sin especificar la clase concreta.
- **Motivación**
  - Un toolkit interfaz de usuario que soporta diferentes plataformas: Sun, Windows, Ubuntu, Mac,..
  - Se desea independencia de la plataforma (cross-platform).



Factoría Abstracta: Motivación



Factoría Abstracta: Motivación



Factoría Abstracta: Ejemplo de uso

```
// al inicio de la ejecución se crea el objeto factoría concreto

WidgetFactory wf = WindowsFactory.getInstancia()

// la factoría permite crear objetos sin indicar la clase

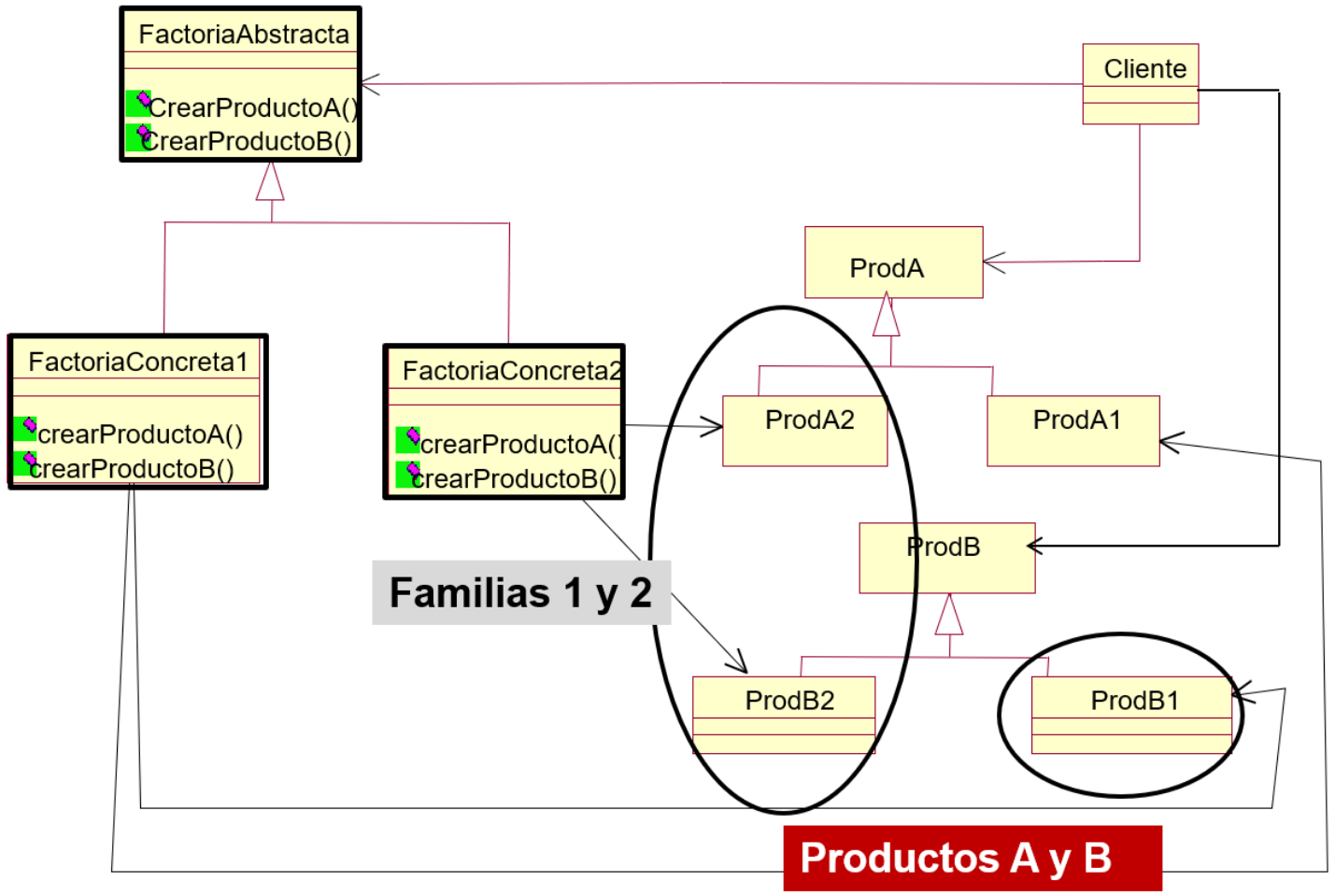
Button boton = wf.createButton();

List lista =  wf.createList();
```

Singleton

Código independiente de la plataforma

Factoría Abstracta: Estructura, Aplicabilidad, Consecuencias



Aplicabilidad

Un sistema debería ser configurado para una familia de productos y ser independiente de familias y productos concretos.

Consecuencias

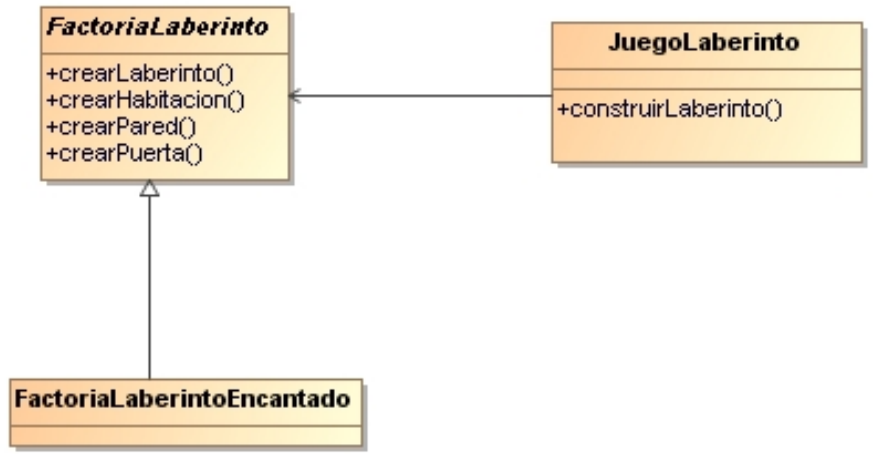
- Desacopla a la aplicación de las clases concretas.
- Facilita el intercambio de familias de productos. No siempre necesario.
- Favorece la consistencia entre productos
- Un nuevo producto requiere modificar todas las clases factorías.

Implementación

- Factorías como singleton.

Factoría abstracta: Ejemplo Laberinto

```
public class FactoriaLaberinto {
    public Laberinto crearLaberinto {
          return new Laberinto(); }
    public Pared  crearPared {
          return new Pared(); };
    public Habitacion  crearHabitacion(int n){
          return new Habitacion(n); };
    public Puerta crearPuerta (Habitacion h1, Habitacion h2) {
          return new Puerta(h1,h2);  }
}
```



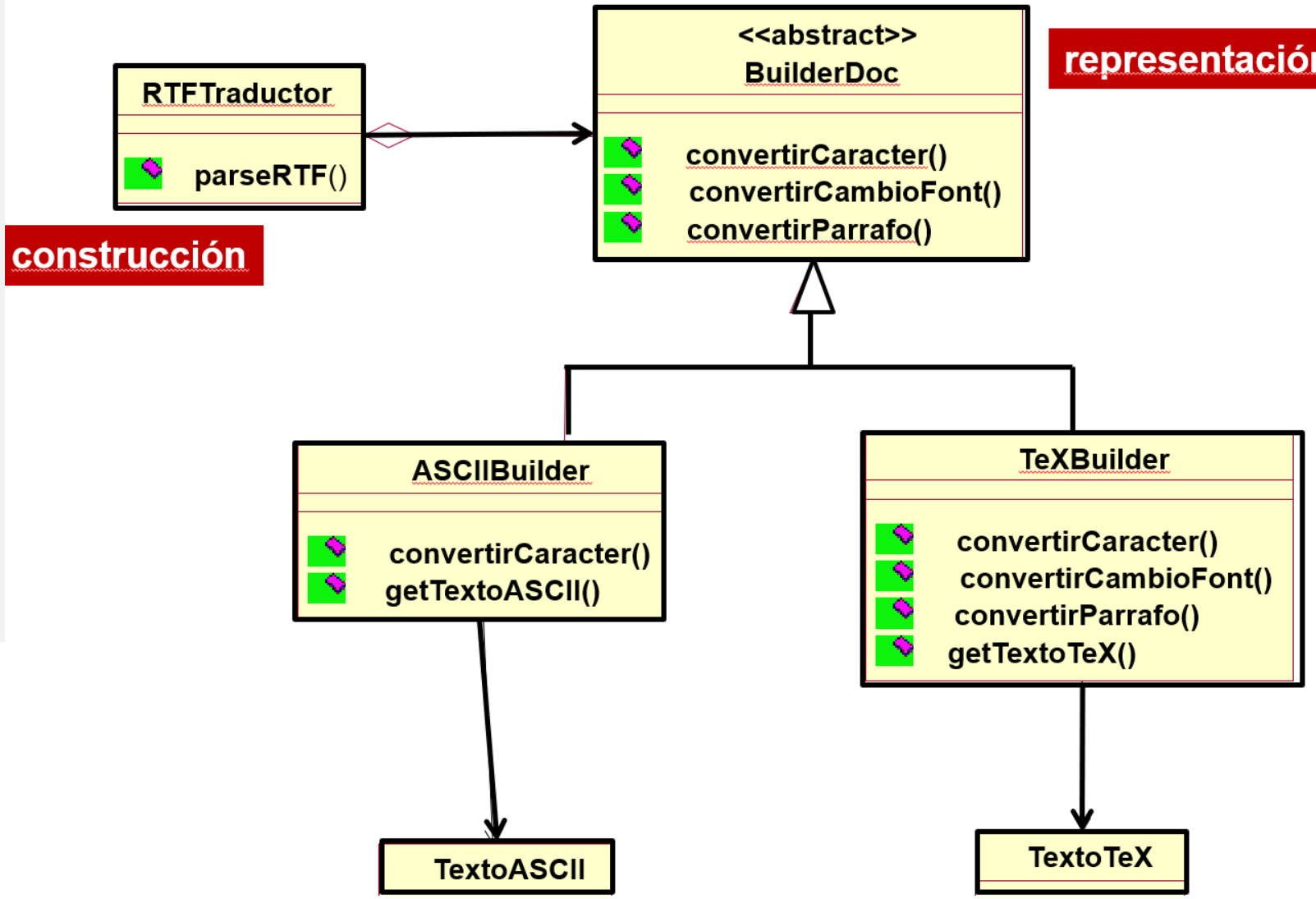

```
public class FactoriaLaberintoEncantado extends FactoriaLaberinto {
    public ParedEncantada  crearPared {
          return new ParedEncantada(); };
    public HabitacionEncantada  crearHabitacion(int n){
          return new HabitacionEncantada(n); };
    public PuertaEncantada crearPuerta (Habitacion h1, Habitacion h2) {
          return new PuertaEncantada(h1,h2); }
}
```

Factoría abstracta: Ejemplo Laberinto

```
public class JuegoLaberinto {
   public Laberinto construirLaberinto (FactoriaLaberinto factoria) {
        Laberinto unLaberinto = factoria.crearLaberinto();
        Habitacion h1 = factoria.crearHabitacion(1);
        Habitacion h2 = factoria.crearHabitacion(2);
        Puerta unaPuerta = factoria.crearPuerta(h1,h2);
        unLaberinto.addHabitacion(h1);
        unLaberinto.addHabitacion(h2);
        h1.setLado(Norte, factoria.crearPared() );33
        h1.setLado(Este, unaPuerta)
        ..
        h2.setLado(Oeste, unaPuerta);
        h2.setLado(Sur, factoria.crearPared() )
        return unLaberinto;
    }
}
```

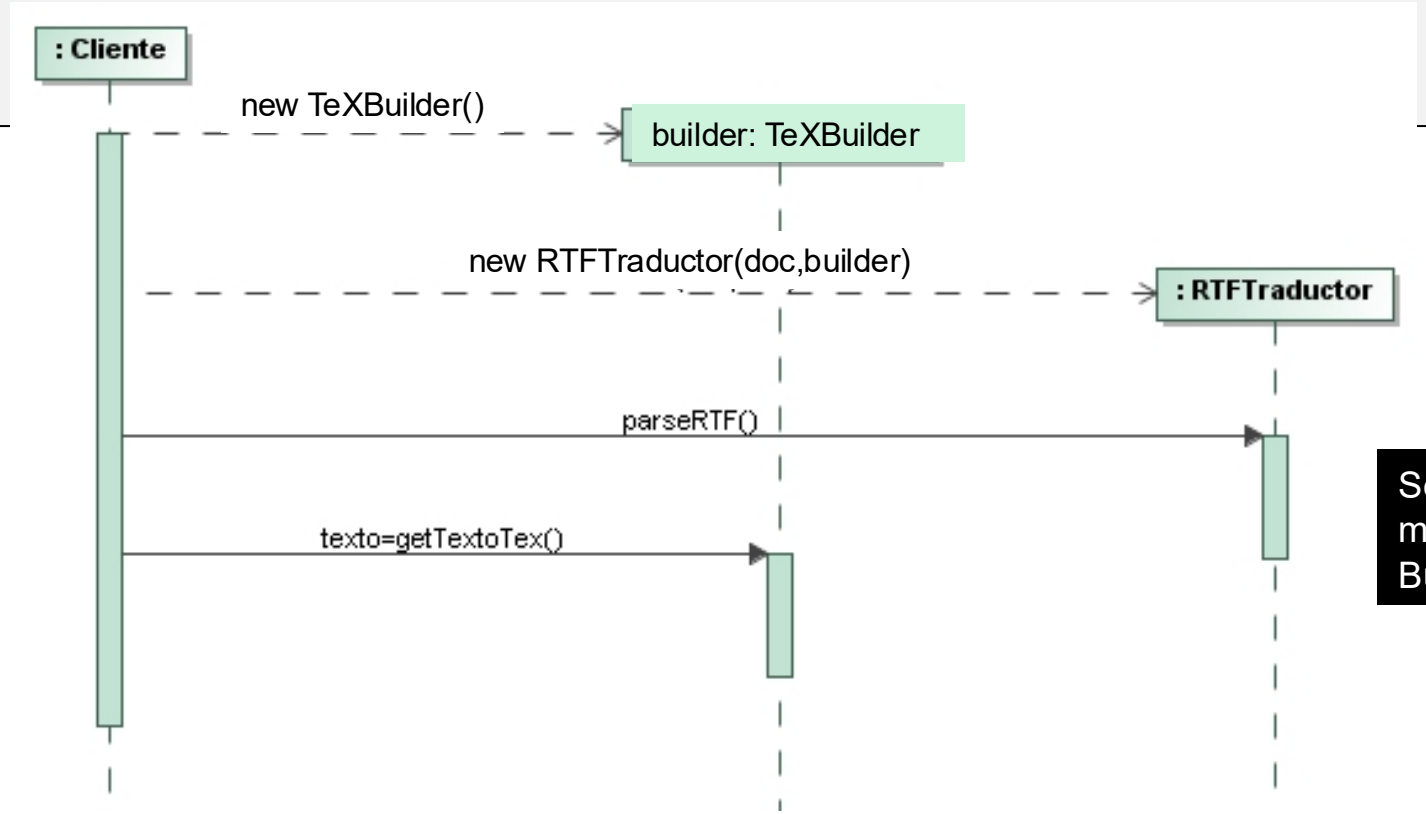
Builder (Constructor)

- **Propósito**
  - Construcción de un objeto complejo, separando el proceso de construcción de su representación, así que el mismo proceso puede crear diferentes representaciones.
- **Motivación**
  - Un traductor de documentos RTF a otros formatos. ¿Es posible añadir una nueva conversión sin modificar el traductor?



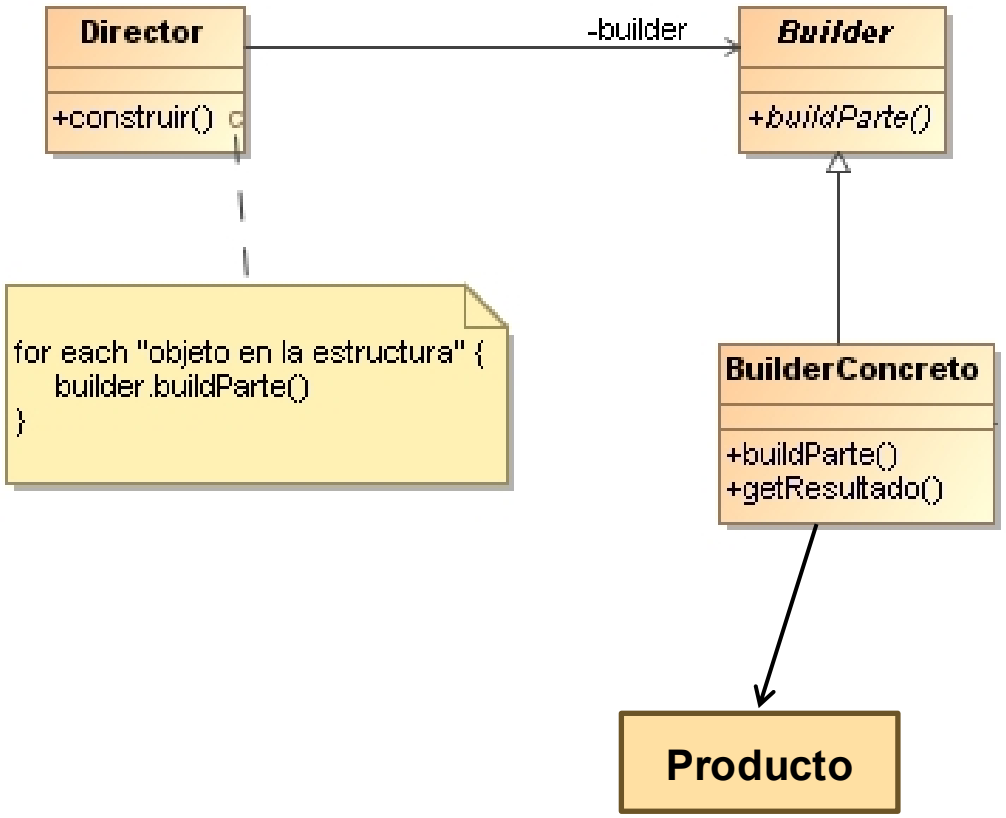
Builder: Motivación

```
// análisis para encontrar "piezas" a ensamblar
// builder y doc inicializados en constructor
void parseRTF() {
   while (token = “obtener siguiente token de texto”) {
      switch(token.tipo) {
         case Car:  builder.convertirCaracter(token.char);break;
         case Font: builder.convertirCambioFont(token.font);break;
         case Par:  builder.convertirParrafo(token.par); break
       }
   }
}
```



Se omiten invocaciones métodos “convertir” del Builder

Builder: Estructura, Aplicabilidad y Consecuencia



Aplicabilidad

- Crear un objeto complejo separando el proceso de construcción de las piezas que lo conforman y de cómo se ensamblan.
- El proceso de construcción debe permitir diferentes representaciones para el objeto que se construye.

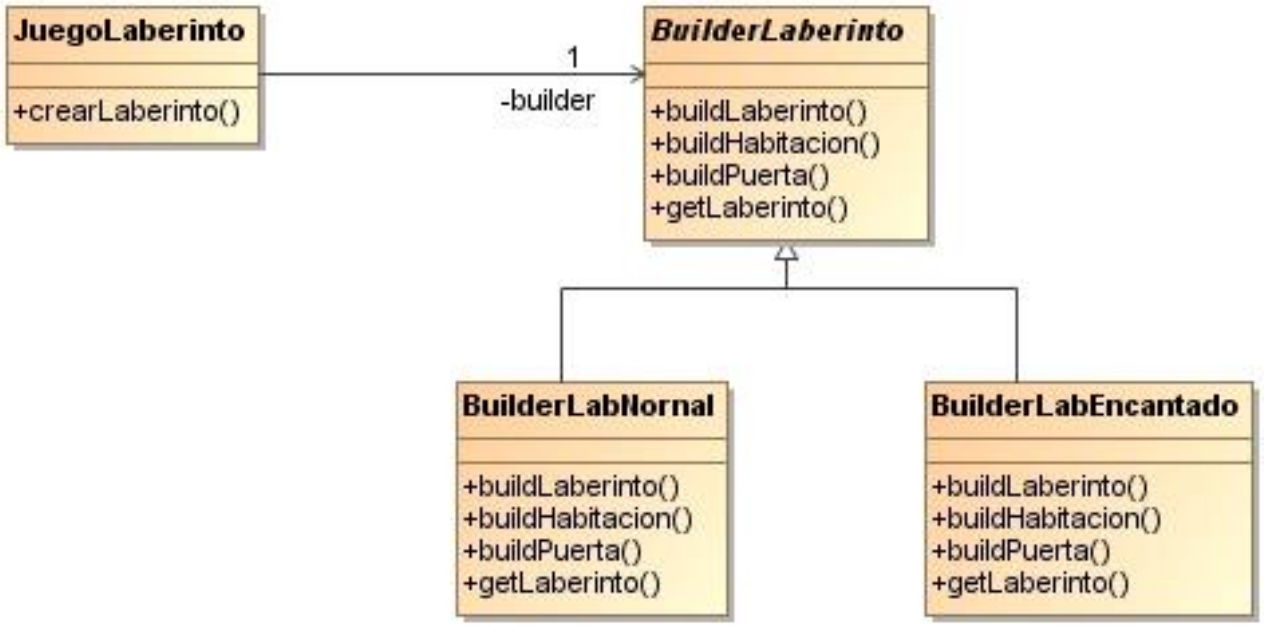
Consecuencias

- Permite cambiar la representación del producto.
- Reúne el código para la representación y ensamblaje.
- Los clientes no necesitan conocer nada sobre la estructura interna.
- Proporciona un control fino del proceso de construcción.
- Diferentes “directores” pueden reutilizar un mismo “builder”

Builder: Implementación

Implementación

- La **interfaz de *Builder* debe ser lo suficientemente general** para permitir la construcción de productos para cualquier Builder concreto.
- La **construcción puede ser más complicada** que añadir el nuevo token al producto en construcción.
- Los métodos de la clase `Builder` **pueden no ser abstractos sino vacíos**.
- Las clases de los **productos no siempre tienen una clase abstracta común**.



Las **"paredes"** forman parte de la representación interna

```
public class BuilderLaberinto {
   public void buildLaberinto () { };
   public void   buildPuerta (int r1, int r2) { };
   public void buildHabitacion(int n) { };
   public Laberinto getLaberinto() { };
   protected BuilderLaberinto() { };
}
```

Builder: Ejemplo Laberinto

Director

```
public class JuegoLaberinto {
    public Laberinto  construirLaberinto (BuilderLaberinto builder){
        builder.buildLaberinto();

        builder.buildHabitacion(1);
        builder.buildHabitacion(2);
        builder.buildPuerta(1,2);
        ..
        return builder.getLaberinto();
    }
}
```




```
public class BuilderLabNormal extends BuilderLaberinto {..
        private Laberinto labActual;
        // se define la construcción de un laberinto con determinado
        // tipo de puertas y habitaciones
}
```

```
public class BuilderLabNormal extends BuilderLaberinto {
   private Laberinto labActual;
   public BuilderLabNormal () {  labActual = null;   }

   public void buildLaberinto ()  { labActual = new Laberinto(); }
   public void buildHabitacion (int n) {
       if ( labActual.numeroHabitacion(n) == null ) {
              Habitacion hab = new Habitacion (n);
              labActual.addHabitacion (hab);
              hab.setLado (Norte, new Pared());
              hab.setLado (Sur, new Pared());
              hab.setLado (Este, new Pared());
              hab.setLado (Oeste, new Pared());
       }
    }
    public void buildPuerta (int n1, int n2)) {
          Habitacion h1 = labActual.numeroHabitacion(n1);
          Habitacion h2 = labActual.numeroHabitacion(n2);
          Puerta p = new Puerta(h1,h2);
          h1.setLado(getParedComun(h1,h2), p);
          h2.setLado(getParedComun(h2,h1), p);
     }
     public Laberinto getLaberinto () {  return labActual;  }
}
```

```
public class BuilderLabNormal extends BuilderLaberinto {
   private Laberinto labActual;
   public BuilderLabNormal () {  labActual = null;   }

   public void buildLaberinto ()  { labActual = new Laberinto(); }
   public void buildHabitacion (int n) {
       if ( labActual.numeroHabitacion(n) == null ) {

BuilderLabNormal builder = new BuilderLabNormal();
JuegoLaberinto juego = new JuegoLaberinto (builder);


Laberinto lab = juego.construirLaberinto(builder);
       }
       public void buildPuerta (int n1, int n2)) {
             Habitacion h1 = labActual.numeroHabitacion(n1);
             Habitacion h2 = labActual.numeroHabitacion(n2);
             Puerta p = new Puerta(h1,h2);
             h1.setLado(getParedComun(h1,h2), p);
             h2.setLado(getParedComun(h2,h1), p);
       }
       public Laberinto getLaberinto () {  return labActual;  }
}
```

“Considera un builder cuando te enfrentes a un constructor con muchos parámetros” (“*Effective Java*”, 2ª Ed., Joshua Bloch)

¡¡ NO CONFUNDIR CON PATRÓN BUILDER !!

```
public class DatosNutricionales {
   private final int tamañoUnidad;       //obligatorio
   private final int unidades;           //obligatorio
   private final int calorias;           //opcional
   private final int grasas;             //opcional
   private final int sodio;              //opcional
   private final int carbohidratos;      //opcional

   public static class Builder {..}
   // clase anidada estática en siguiente diapositiva

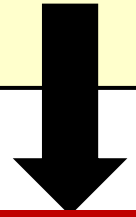
   private DatosNutricionales (Builder builder) {
         tamañoUnidad = builder.tamañoUnidad;
         unidades = builder.unidades;
         calorias= builder.calorias;
         grasa= builder.grasas;
         sodio = builder.sodio;
         carbohidratos = builder.carbohidratos;
   }
}
```

Muchos campos opcionales

```java
public static class Builder {
    private final int tamañoUnidad;
    private final int unidades;
    private final int calorias = 0;
    private final int grasas = 0;
    private final int sodio = 0;
    private final int carbohidratos = 0;

    public Builder (int tamañoUnidad, int unidades) {
         this.tamañoUnidad = tamañoUnidad;
         this.unidades = unidades;
    }
    public Builder calorias (int val) {
         calorias = val; return this;
    }
    public Builder grasas(int val) {
         grasas= val; return this;
    }
    public Builder sodio(int val) {
         sodio= val; return this;
    }
    public Builder carbohidratos(int val) {
         carbohidratos= val; return this;
    }
    public DatosNutricionales build () {
         return new DatosNutricionales(this);}
}
```

```java
DatosNutricionales cocacola =
    new DatosNutricionales.Builder(240,8)
      .calorias(100)
      .sodio(35)
      .carbohidrato(27)
      .build();
```



Facilidad de uso

Fluent-API basada en técnica “method chaining”

Factory Method (Método Factoría)

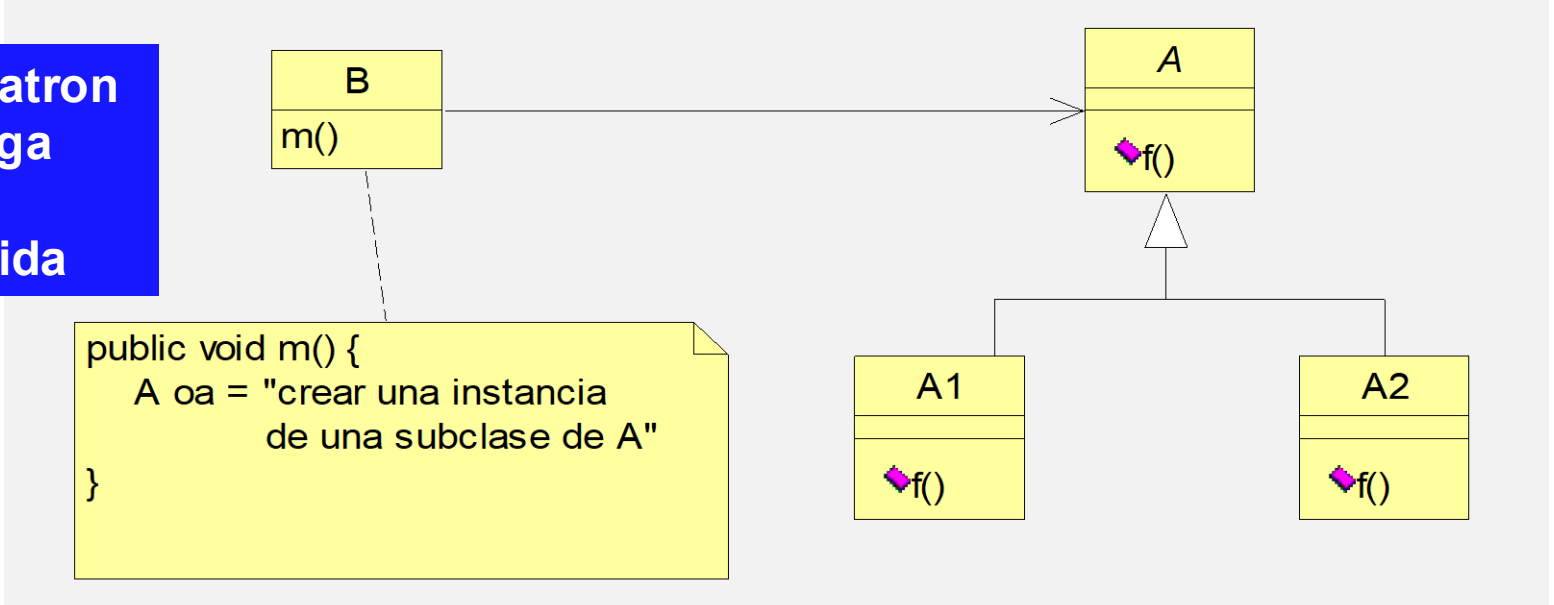
Propósito

- Define un interfaz para crear un objeto, pero permite a las subclases decidir la clase a instanciar: ***instanciación diferida a las subclases.***

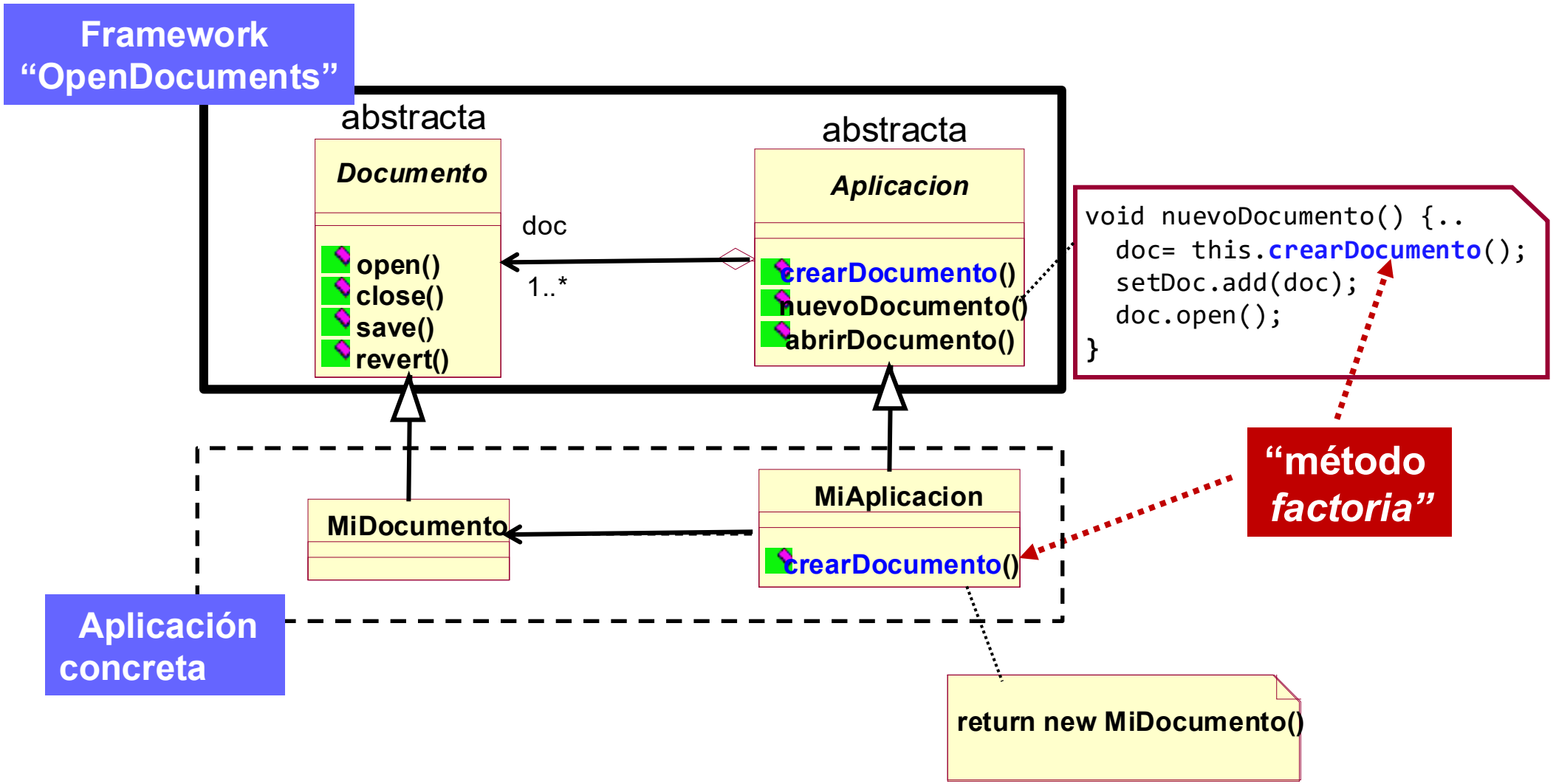
Motivación

- Una clase `B` que usa una clase abstracta `A` necesita crear instancias de subclases de `A` que no conoce.
- En un *framework* para aplicaciones que pueden presentar al usuario documentos de distinto tipo: clases `Aplicación` y `Documento`.
- Se conoce **cuándo** crear un documento, pero no se conoce **de qué tipo**.

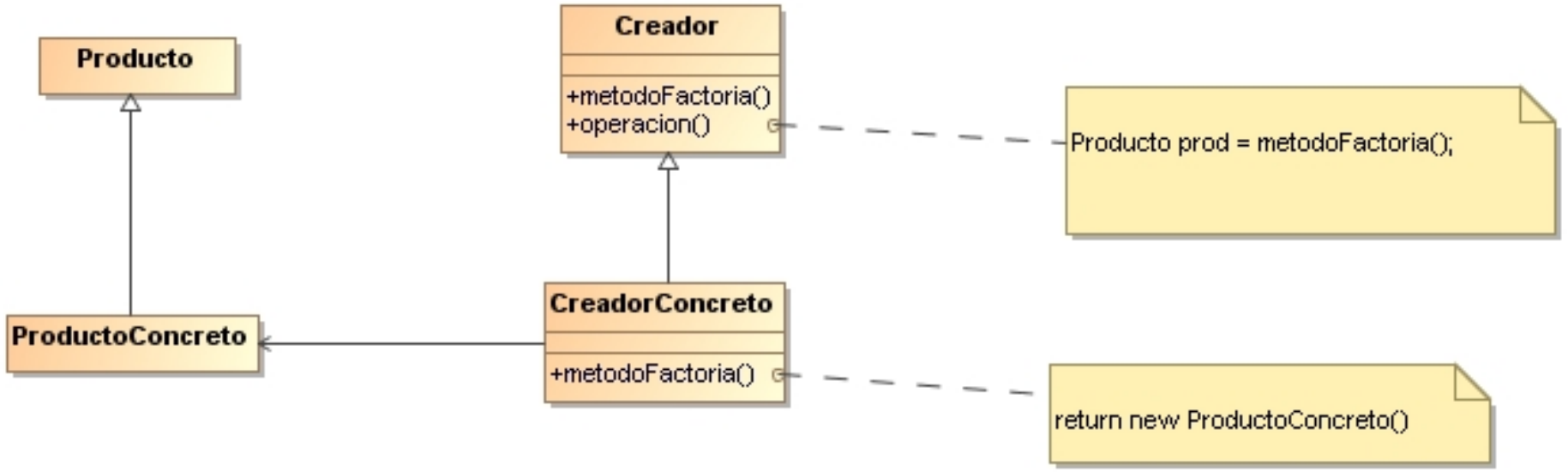
El enunciado del patron requiere que B tenga subclases para la instanciación diferida



Método Factoría: Motivación



Método Factoría: Estructura, Aplicabilidad, Consencuencias



Aplicabilidad

Una clase no puede anticipar la clase de objetos que debe crear.

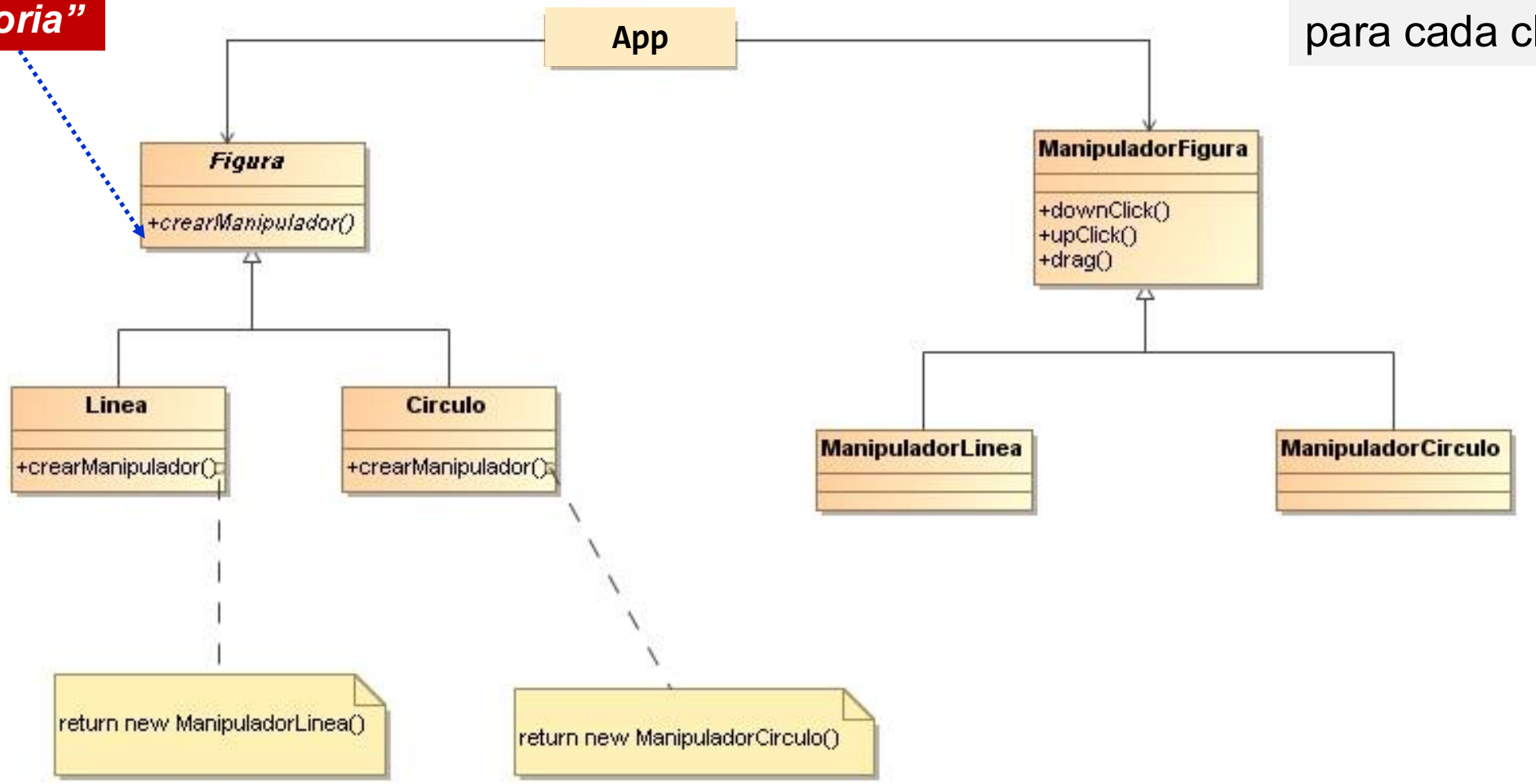
Consecuencias

- Evita ligar un código a clases específicas de la aplicación.
- Puede suceder que las subclases de ***Creador*** sólo se creen con el fin de la creación de objetos - - > **Evitar**
- El método factoría puede ser invocado por un cliente, no sólo por la clase *Creador* - - > **jerarquías paralelas**

Método Factoría: Jerarquías paralelas

"método *factoria*"

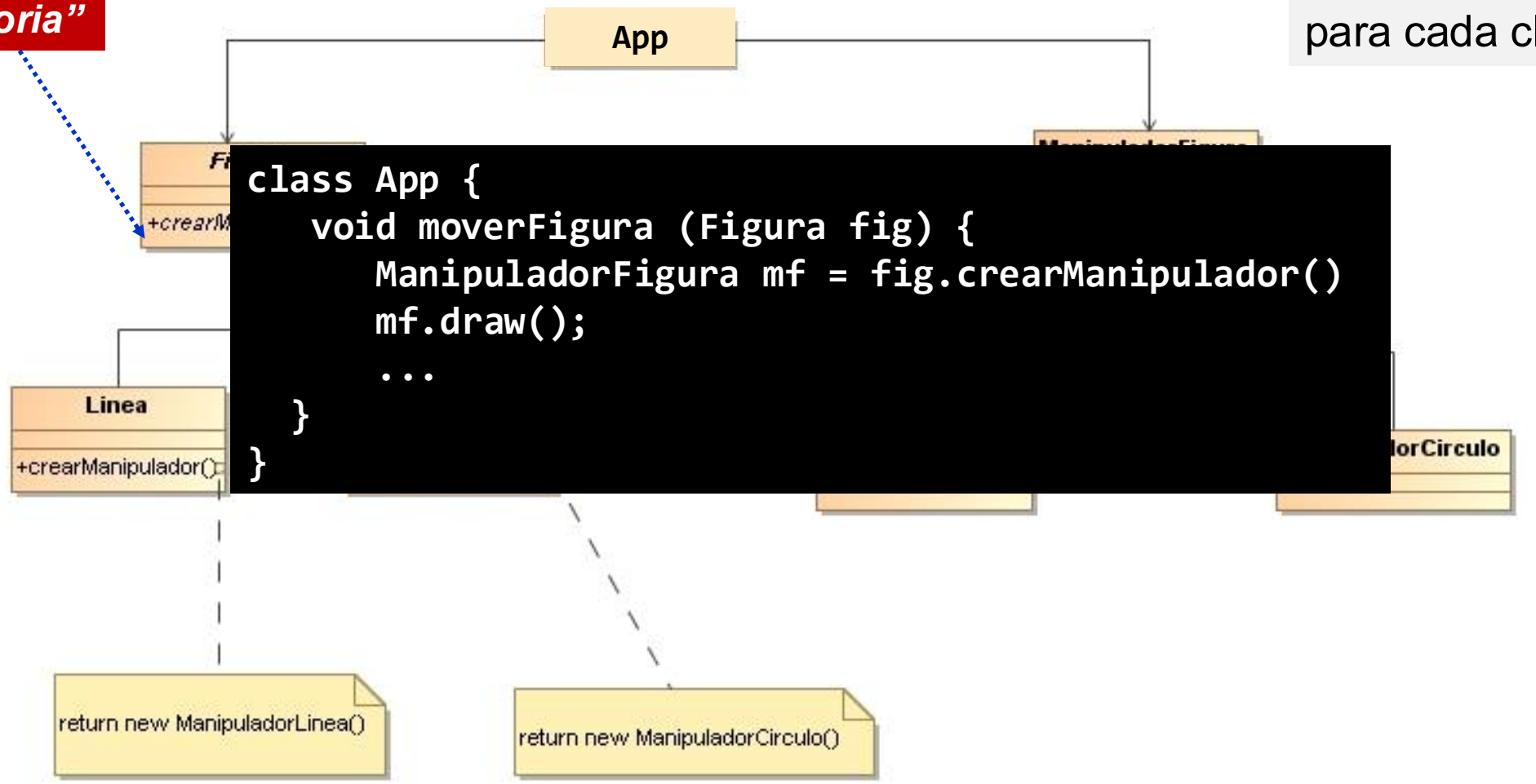
Existe una clase manipulador para cada clase de figura



Método Factoría: Jerarquías paralelas

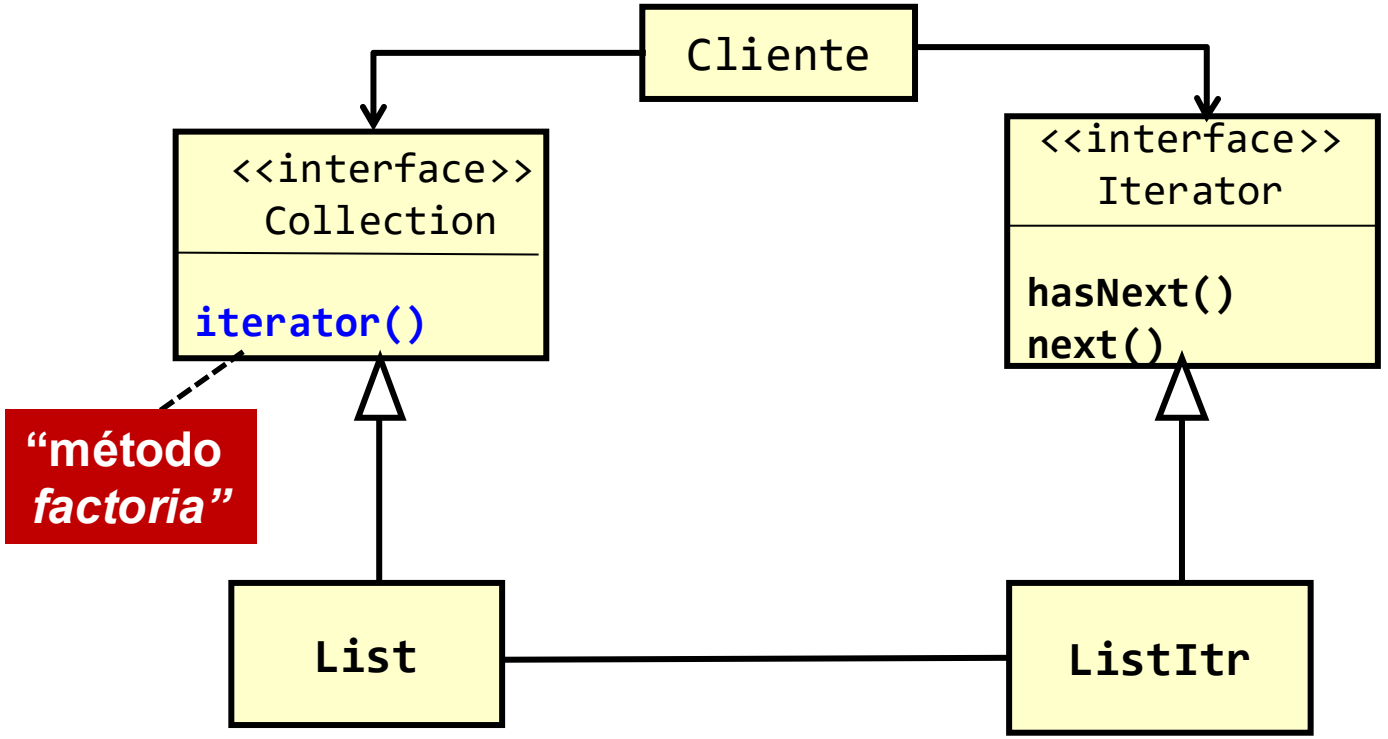
"método *factoria*"

Existe una clase manipulador para cada clase de figura



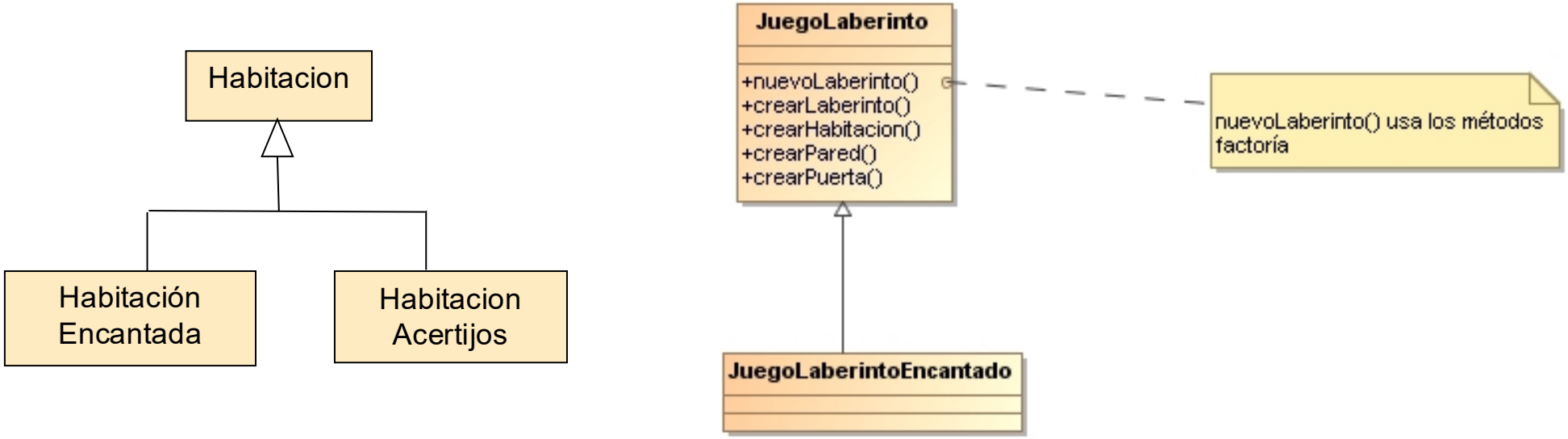

```
class App {
   void moverFigura (Figura fig) {
      ManipuladorFigura mf = fig.crearManipulador()
      mf.draw();
      ...
  }
}
```

Método Factoría: Jerarquías paralelas en caso de un iterador externo en Java




```
class Cliente{
   void printCollection (Collection c) {
        Iterator it = c.iterator();
        while (it.hasNext()) {
             System.out.
                  println(it.next())
   }
}
```

Método Factoría: Ejemplo Laberinto




```
public class JuegoLaberinto {
   public Laberinto  construirLaberinto () {...};

   public Laberinto crearLaberinto() { return new Laberinto(); };
   public Habitacion crearHabitacion(int n)
        { return new Habitacion(n); };
   public Pared crearPared() { return new Pared(); };
   public Puerta crearPuerta(Habitacion h1, Habitacion h2)
              { return new Puerta(h1,h2); };
}
```

Método Factoría: Ejemplo Laberinto

```
public Laberinto construirLaberinto () {
        Laberinto unLab = this.crearLaberinto();
        Habitacion h1 = this.crearHabitacion(1);
        Habitacion h2 = this.crearHabitacion(2);
        Puerta unaPuerta = this.crearPuerta(h1,h2);
        unLab.addHabitacion(h1);
        unLab.addHabitacion(h2);
        h1.setLado(Norte, crearPared() );
        h1.setLado(Este, unaPuerta)
        ..
        h2.setLado(Oeste, unaPuerta);
        h2.setLado(Sur, crearPared() );
        return unLab; }
}
```

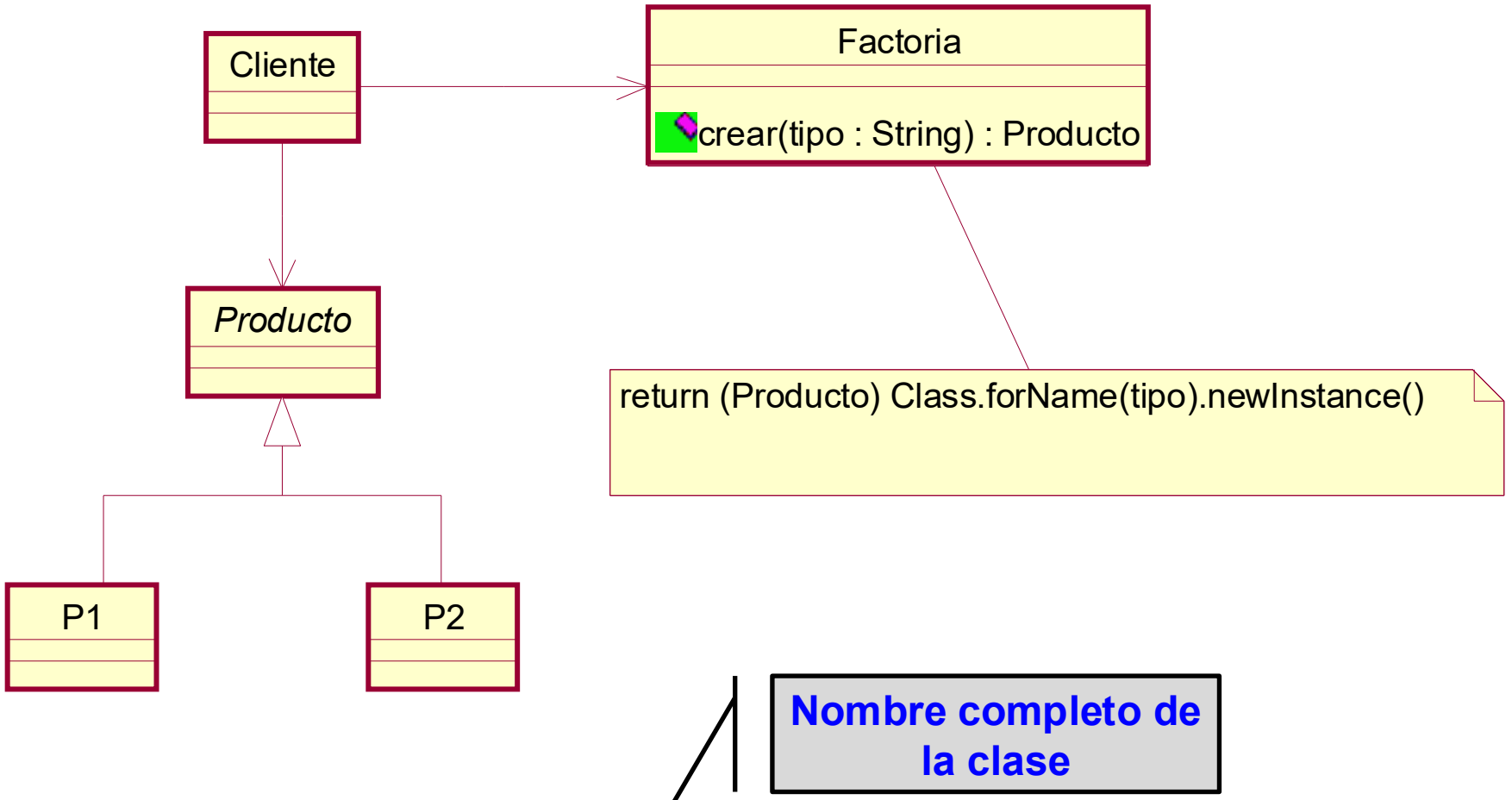
Método Factoría: Ejemplo Laberinto

```
public class JuegoLaberintoEncantado extends JuegoLaberinto {
        public HabitacionEncantada crearHabitacion(int n) {
            return new HabitacionEncantada(n);
        }
        public ParedEncantada crearPared() {
            return new ParedEncantada();
        }
        public PuertaEncantada crearPuerta(Habitacion h1,
                                           Habitacion h2) {
            return new PuertaEncantada(h1, h2);
        }
}
```

Método Factoría: Implementación

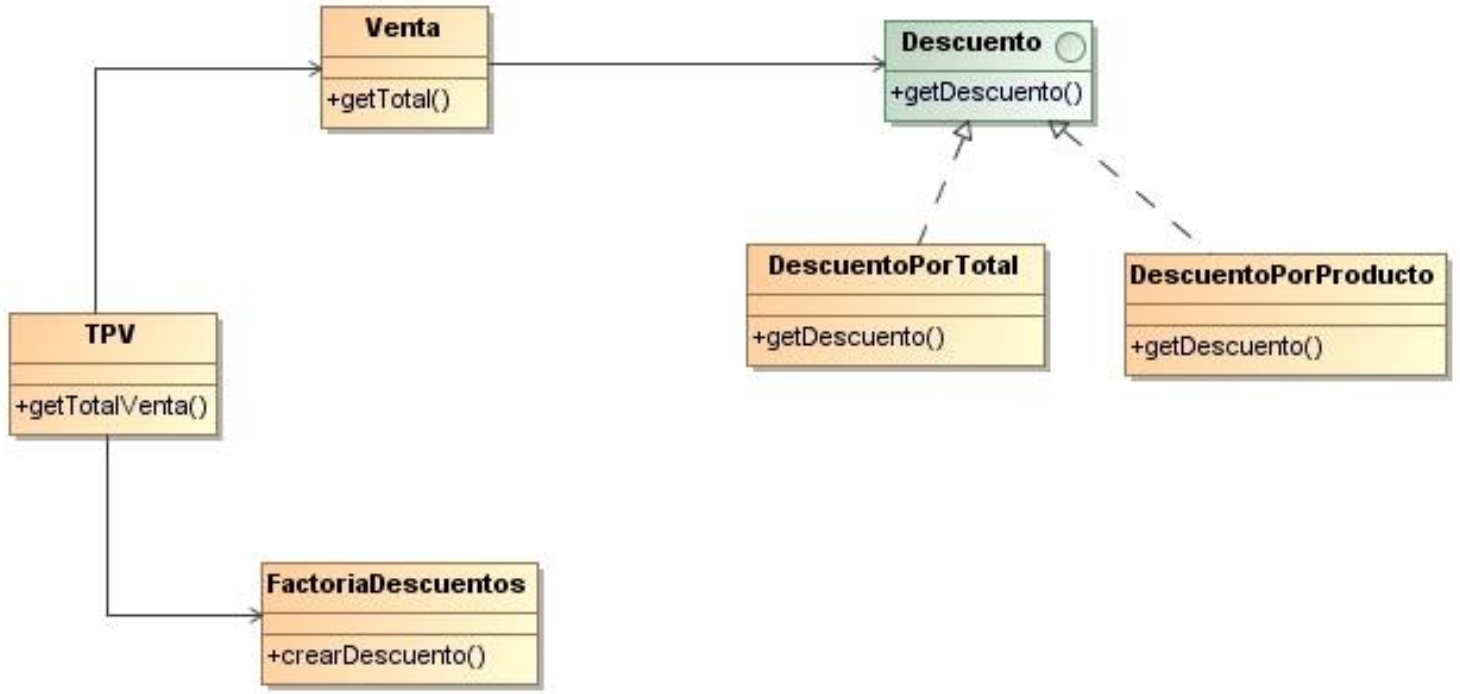
- Dos posibilidades para **implementar el método factoría** en la clase raíz de la jerarquía de productos
  - Método abstracto que se implementa en las subclases
  - Una implementación por defecto.
- **Cuando no hay subclases de Creador**, se evita tener que crearlas con una **clase Factoría** que contiene el método factoría que tiene un parámetro que indica qué tipo de objeto crear. Varias posibles implementaciones
  - Switch
  - Uso de Introspección (`Class<>.forname().newInstance en Java`)
  - Interface funcional `Supplier<>`

Método Factoría con newInstance()




```
Class<?>.forName(tipoProducto).getDeclaredConstructor()
                              .newInstance()
```

Ejemplo de Método Factoría con `newInstance()`




```
public Descuento crearDescuento(String tipoDescuento){
    Class<?>.forName(tipoDescuento).getDeclaredConstructor().newInstance()
}
```

```
public class TPV {
  public double getTotalVenta() {
    Descuento  desc= FactoriaDescuentos.getInstance().crearDescuento(tipo);
    ventaActual.getTotal(desc);
    …
  }
```

The Factory Pattern Using Lambda Expressions in Java 8

The factory pattern is one of the best known patterns in Java. If you're using lambda expressions, you can use those to implement the pattern, though beware scaling.



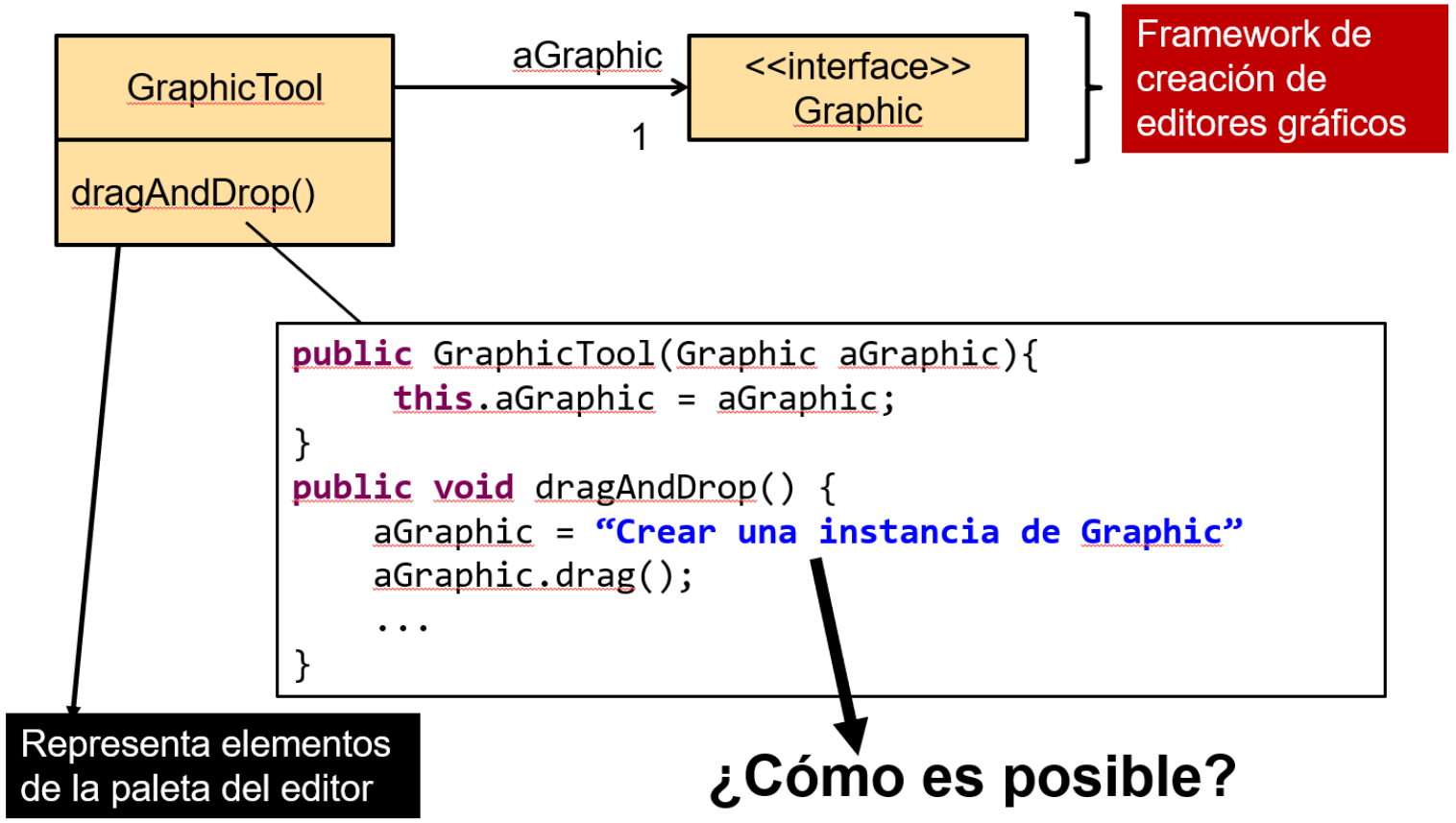
by Monika Goel · Mar. 01, 17 · Java Zone · Tutorial

```java
public class ShapeFactory {

  final static Map<String, Supplier<Shape>> map = new HashMap<>();
  static {
    map.put("CIRCLE", Circle::new);
    map.put("RECTANGLE", Rectangle::new);
  }
  public Shape getShape(String shapeType){
     Supplier<Shape> shape = map.get(shapeType.toUpperCase());
     if(shape != null) {
       return shape.get();
     }
     throw new IllegalArgumentException("No such shape " + shapeType.toUpperCase());
  }
}
```

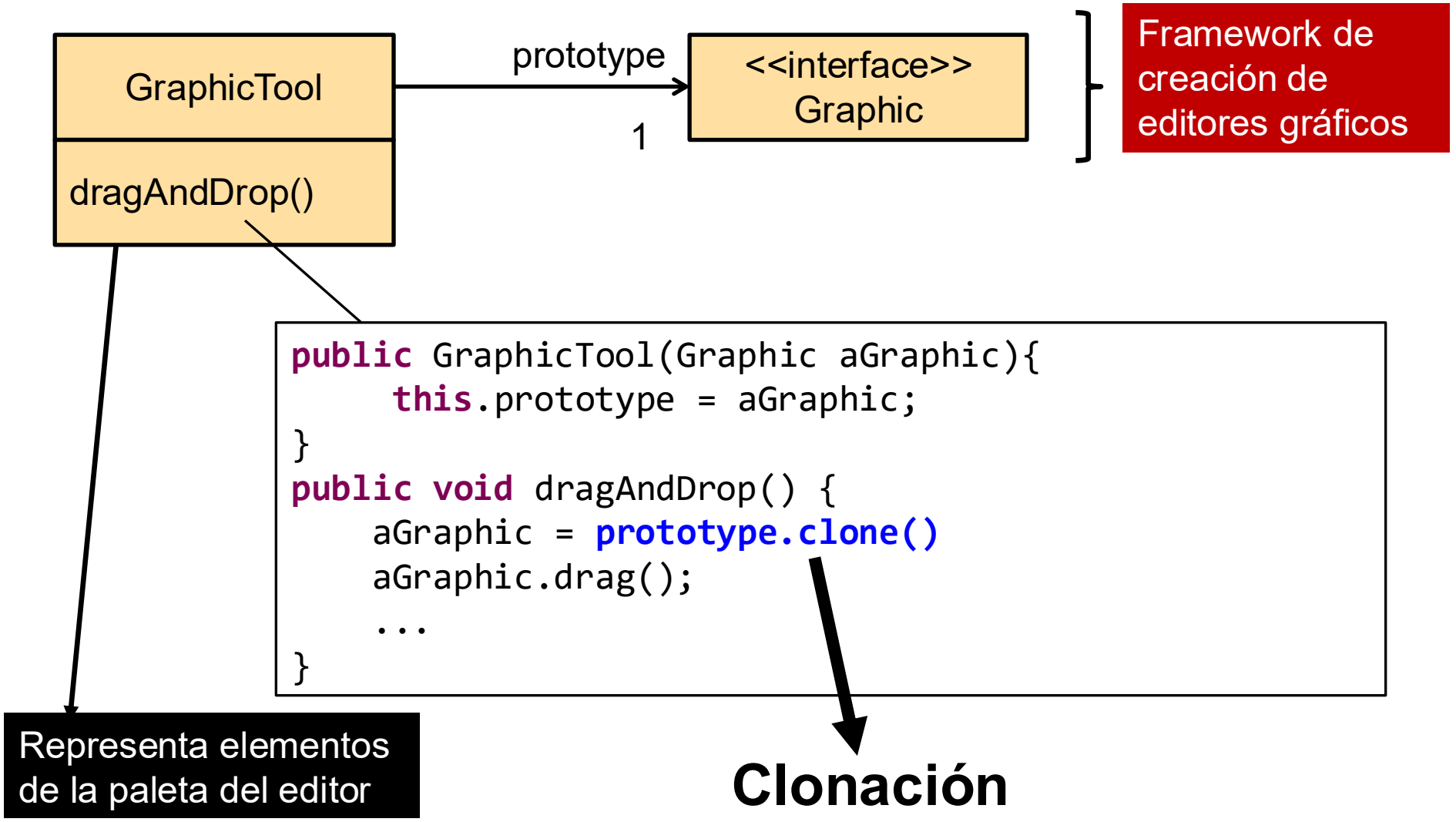
Ejemplo en el que no es apropiado aplicar el Método Factoría.
Motivación del libro "Design Patterns" de Gamma et al. para el patron *Prototipo.*

- Sea un framework para crear editores gráficos que incluye las clases `Graphic` y `GraphicTool`.
- `GraphicTool` se utiliza para crear herramientas de la paleta de herramientas. ¿Cómo se asocia a un gráfico concreto y cada vez que se use para añadir un element al panel de edición se cree un diferente elemento del tipo correspondiente?
- Si se aplica *Método Factoría* se deberían crear muchas subclases de `GraphicTool`.



Solución con Prototipo

Motivación del libro “Design Patterns” de Gamma et al. para el patron *Prototipo*



Solución con *Supplier*

```
import java.util.function.Supplier;
public class GraphicTool {
     private Supplier<Graphic> supplier;

     public void setSupplier(Supplier<Graphic> supplier) {
        this.supplier = supplier;
     }
     public void dragAndDrop() {..
        supplier.get().drag();
     }
     public void printGraphicName(){
        System.out.println(supplier.get().getName());
     }
}
public interface Graphic {
    public String getName();
}
```

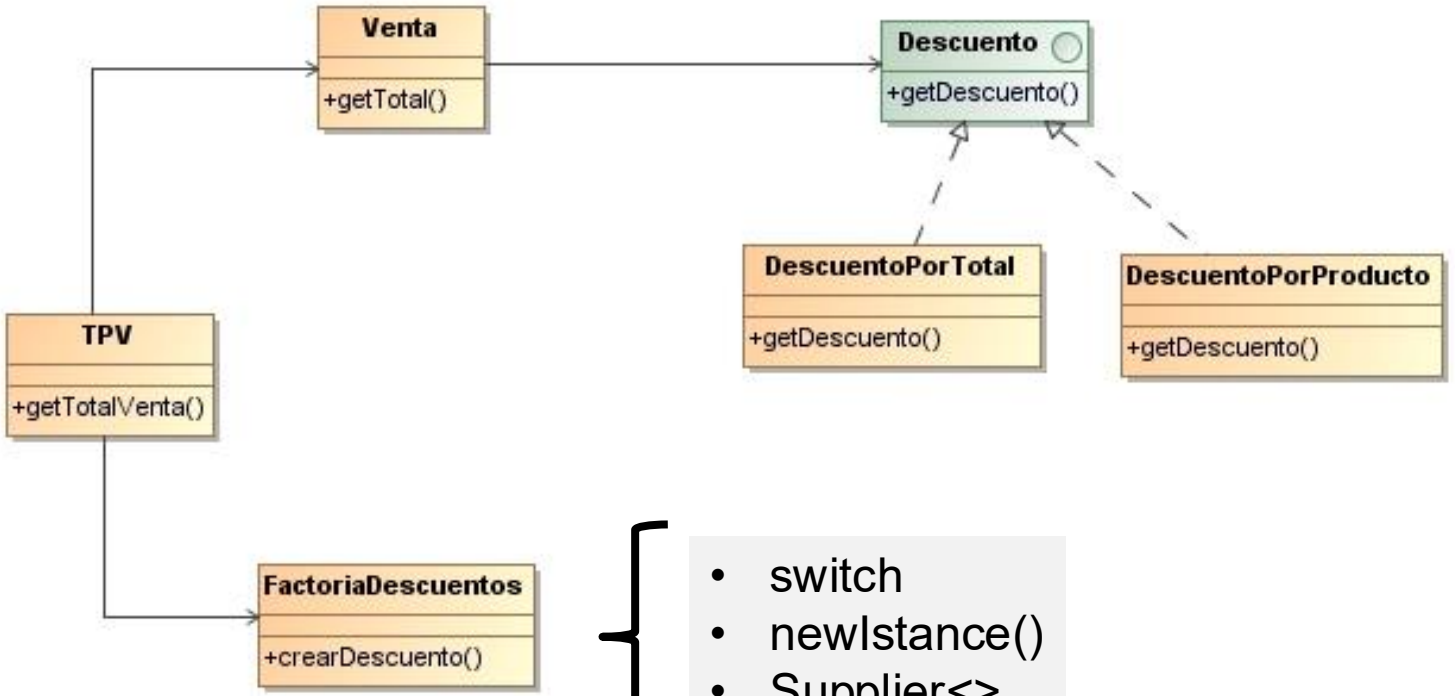
Solución con *Supplier*

```
public class App {

   public static void main(String[] args) {
        Supplier<Graphic> g = () -> new Rectangle("Caja 1",3,2);
        //se crea e inicializa un GraphicTool para rectángulos
        GraphicTool gt = new GraphicTool();
        gt.setSupplier(g);

        gt.printGraphicName();
  }
}
```

Método Factoría: Ejemplo



- switch
- newIstance()
- Supplier<>

Factoría con análisis de casos con `switch`: Ejemplo

```
public class FactoriaDescuentos {

    public Descuento crearDescuento(String tipoDescuento) {
        switch (tipoDescuento.toLowerCase()) {
            case "total":
                return new DescuentoPorTotal();
            case "producto":
                return new DescuentoPorProducto();
            default:
                throw new IllegalArgumentException("Tipo de descuento no soportado:
                                                    " + tipoDescuento);
        }
    }
}
```

Factoría con `newInstance()`: Ejemplo

```
public class FactoriaDescuentos {

    public Descuento crearDescuento(String tipoDescuento) {
        try {
            // Obtener el nombre de clase completo a partir del tipo de descuento
           // Asegurar de tener el paquete correcto
            String nombreClase = "paquete.descuentos." + tipoDescuento;
            Class<?> clase = Class.forName(nombreClase);
            return (Descuento) clase.getDeclaredConstructor().newInstance();
        } catch (Exception e) {
              throw new IllegalArgumentException("Error al crear el descuento: " +
                                                                    tipoDescuento, e);
        }
    }
}
```

Factoría con Supplier<>: Ejemplo

```
public class FactoriaDescuentos {
    // Map de tipos de descuento y sus suppliers
    private static final Map<String, Supplier<Descuento>> descuentoMap = new HashMap<>();
    static {
        // Registrar los suppliers en el map
        descuentoMap.put("total", DescuentoPorTotal::new);
        descuentoMap.put("producto", DescuentoPorProducto::new);
    }
    public Descuento crearDescuento(String tipoDescuento) {
        Supplier<Descuento> descuentoSupplier =
                                descuentoMap.get(tipoDescuento.toLowerCase());
        if (descuentoSupplier != null) {
            return descuentoSupplier.get();
        } else {
            throw new IllegalArgumentException("Tipo de descuento no soportado: " +
                                                                    tipoDescuento);
        }
    }
}
```

The Factory Pattern Using Lambda Expressions in Java 8

The factory pattern is one of the best known patterns in Java. If you're using lambda expressions, you can use those to implement the pattern, though beware scaling.



by Monika Goel · Mar. 01, 17 · Java Zone · Tutorial

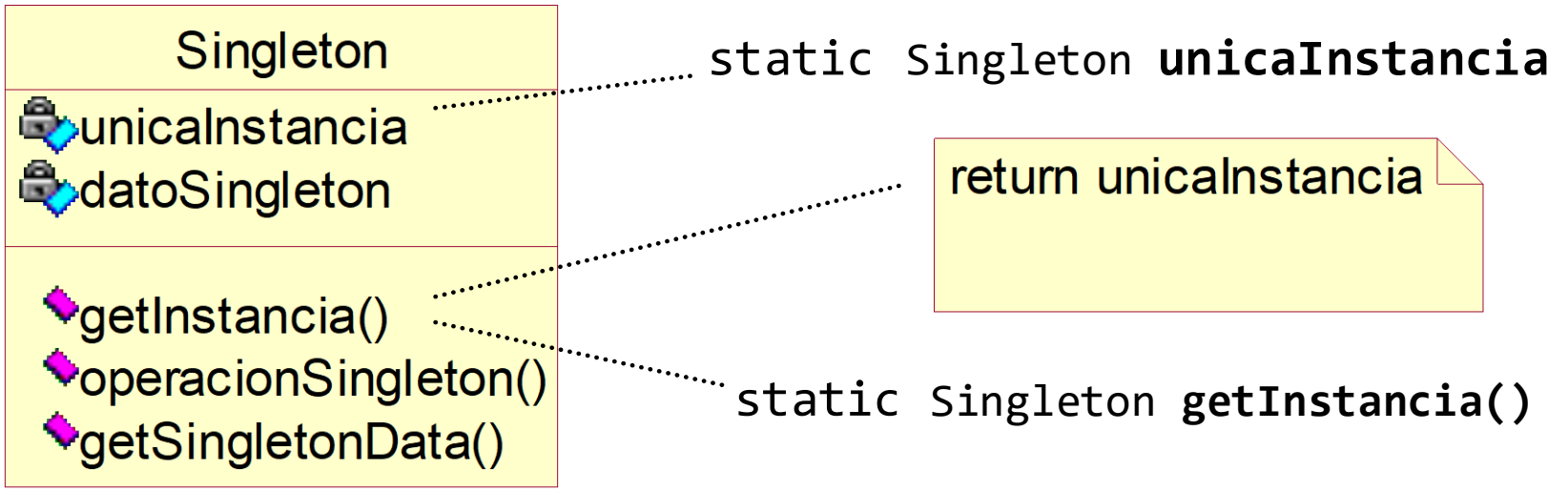
```java
public class ShapeFactory {

  final static Map<String, Supplier<Shape>> map = new HashMap<>();
  static {
    map.put("CIRCLE", Circle::new);
    map.put("RECTANGLE", Rectangle::new);
  }
  public Shape getShape(String shapeType){
     Supplier<Shape> shape = map.get(shapeType.toUpperCase());
     if(shape != null) {
       return shape.get();
     }
     throw new IllegalArgumentException("No such shape " + shapeType.toUpperCase());
  }
}
```

Singleton

- **Propósito**
  - Asegurar que una clase tiene **una única instancia** y asegurar un punto de **acceso global**.
- **Motivación**
  - Ejemplos: un catálogo o repositorio, una factoría abstracta, un manejador de ventanas,...
  - La clase se encarga de asegurar que exista una única instancia y de su acceso.

`Singleton.getInstancia().operacionSingleton()`



Singleton: Inicialización perezosa

```
public class Singleton {
   private static Singleton unicaInstancia = null;

   public static Singleton getInstancia() {
        if (unicaInstancia == null)  unicaInstancia = new Singleton();
        return unicaInstancia;
   }
   private Singleton() { }

   private int dato = 0;
   public int getDato() {
        return dato;
   }
   public void setDato(int i) {dato = i;}
}
```

1

2

3

clase “normal”

Singleton: Inicialización temprana

```
public class Singleton {
      private static Singleton unicaInstancia = new Singleton();
       private Singleton() { }
       public static Singleton getInstancia() {
              return unicaInstancia;
        }
      private int dato = 0;
      public int getDato() {return dato;}
       public void setDato(int i) {dato = i;}
}
```

Inconvenientes:

- Siempre se crea la instancia, aunque luego no se necesite.
- La inicialización puede necesitar lógica de creación compleja y condicional

Singleton: Aplicabilidad y Consecuencias

- **Aplicabilidad**
  - Debe existir una única instancia de una clase, accesible globalmente.
- **Consecuencias**
  - Acceso controlado a la única instancia
  - Evita usar variables globales
  - Es posible generalizar a un número variable de instancias
  - **No es lo mismo que declarar todos los métodos estáticos**
    - No es posible definir una jerarquía de clases
    - Una clase puede cambiar y dejar de ser Singleton
  - La clase Singleton puede tener subclases (lo veremos más adelante).

Singleton: Ejemplo “Repositorio de circuitos”

```
public class RepositorioCircuitos {

     private HashMap<String,Circuito> elems;
     private static RepositorioCircuitos unicaInstancia = null;

     public static RepositorioCircuitos getInstancia() {
            if (unicaInstancia == null)
                      unicaInstancia = new RepositorioCircuitos();
            return unicaInstancia;
     }
     private RepositorioCircuitos() {
             elems = new HashMap<String,Circuito>();
     }
     public Circuito getCircuito(String id) {
                return elems.get(id);
     }
     public void addCircuito(String id, Circuito c) {
                 elems.put(id,c);
     }
}
```

```
Circuito sumador = RepositorioCircuitos.getInstancia().getCircuito(“sumador”);
```

Tipo *enum* en Java

```
public enum Planeta {
     MERCURIO (3.303e+23, 2.4397e6),
     VENUS   (4.869e+24, 6.0518e6),
     TIERRA (5.976e+24, 6.37814e6),
     MARTE (6.421e+23, 3.3972e6),
     JUPITER (1.9e+27,   7.1492e7),
     SATURN0  (5.688e+26, 6.0268e7),
     URANO  (8.686e+25, 2.5559e7),
     NEPTUN0 (1.024e+26, 2.4746e7);

     private final double masa;
     private final double radio;
     Planet(double masa, double radio) {
          this.masa = masa;
          this.radio = radio;  }
     private double masa() {
          return masa; }
     private double radio() {
          return radio; }
     public static final double G = 6.67300E-11;
     double gravedadSuperficie() {
          return G * mass / (radius * radius);
}
```

Únicas instancias de la clase Planeta

```
for (Planeta p : Planeta.values()) {
   System.out.printf(
        "Su peso es:",
        p.gravedadSuperficie());
}
```

Singletons con enums

```
public enum RepositorioCircuitos {
     INSTANCE;
     private Hashtable elems;
     private RepositorioCircuitos() {
                 elems = new HashTable();
     }
     public Circuito getCircuito(String id) {
                Circuito c = (Circuito) elems.get(id);
                return (Circuito) c.clone();
     }
     public void addCircuito(String id, Circuito c) {
                  elems.put(id,c);
     }
}
```

```
Circuito sumador = RepositorioCircuitos.INSTANCE.getCircuito("sumador");
```

Effective Java (2ª edición)

Item 3: Enforce the singleton property with a private constructor or an enum type

While this approach has yet to be widely adopted,
a single-element enum type is the best way to implement a singleton.

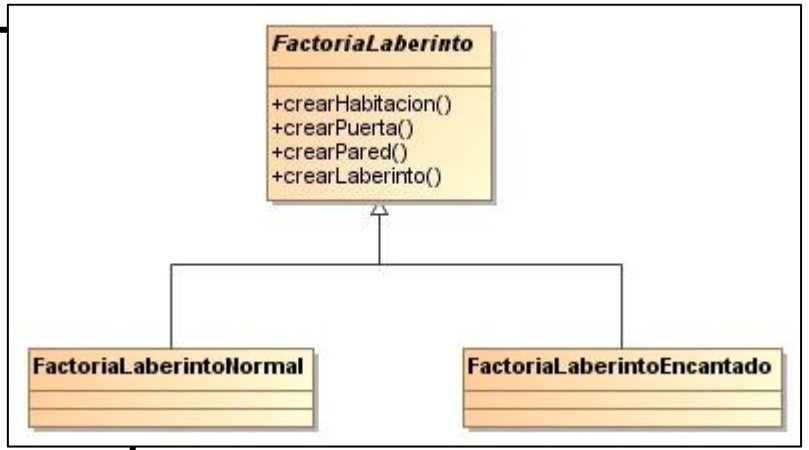
Pero, enums no pueden ser extendidos

Singleton con subclases: Análisis de casos

```
public abstract class FactoriaLaberinto {..
   private static FactoriaLaberinto unicaInstancia = null;

   public static FactoriaLaberinto getInstancia() {
        if (unicaInstancia == null) return getInstancia (“encantado");
        else return unicaInstancia;
   }

      // invocar sólo la primera vez
     public static FactoriaLaberinto getInstancia(String nombre) {
         if (unicaInstancia == null)
                 if (nombre.equals(“encantado"))
                         unicaInstancia = new FactoriaLaberintoEncantado();
                 else if (nombre.equals(“normal"))
                          unicaInstancia = new FactoriaLaberintoNormal();
         return unicaInstancia;
     }
     protected FactoriaLaberinto () { }
}
```



Singleton con subclases: Introspección

```
public abstract class FactoriaLaberinto {..
   private static FactoriaLaberinto unicaInstancia = null;
   public static FactoriaLaberinto getInstancia() {
        if (unicaInstancia == null) return getInstancia (“encantado");
        else return unicaInstancia;
   }
   protected FactoriaLaberinto () { }

   // invocar sólo la primera vez
   public static FactoriaLaberinto getInstancia (String nombre)  {
        if (unicaInstancia == null)
                  unicaInstancia = Class<?>.forName(nombre).
                                   getDeclaredConstructor().newInstance();
        return unicaInstancia ;
   }
}
```

Singleton con subclases: creación en subclases

Cada subclase proporciona un método `getInstancia()`. La primera vez se invoca al método de la subclase que corresponda, el resto de las veces al de la clase abstracta.

```
public abstract class FactoriaLaberinto {..
     protected static FactoriaLaberinto unicaInstancia = null;
     public static FactoriaLaberinto getInstancia() {
              return unicaInstancia;
     }
     protected FactoriaLaberinto () { }
}
```

```
public class FactoriaLaberintoEncantado extends FactoriaLaberinto {
       public static FactoriaLaberinto getInstancia() {
              if (unicaInstancia == null)
                     unicaInstancia = new FactoriaLaberintoEncantado();
              return unicaInstancia;
       }
       private FactoriaLaberintoEncantado() { }
}
```

Singleton: Múltiples hilos

JournalDev

4. Thread Safe Singleton

The easier way to create a thread-safe singleton class is to make the global access method synchronized, so that only one thread can execute this method at a time. General implementation of this approach is like the below class.

```
package com.journaldev.singleton;

public class ThreadSafeSingleton {

    private static ThreadSafeSingleton instance;

    private ThreadSafeSingleton(){}

    public static synchronized ThreadSafeSingleton getInstance(){
        if(instance == null){
            instance = new ThreadSafeSingleton();
        }
        return instance;
    }

}
```

Singleton: Múltiples hilos

JournalDev

The following code snippet provides the double-checked locking implementation.

```
public static ThreadSafeSingleton getInstanceUsingDoubleLocking()
{
    if(instance == null){
        synchronized (ThreadSafeSingleton.class) {
            if(instance == null){
                instance = new ThreadSafeSingleton();
            }
        }
    }
    return instance;
}
```

JournalDev

Previous Next Faceboo

Design Pattern Tutorials

1. Java Design Patterns
2. Singleton
3. Factory
4. Abstract Factory
5. Builder
6. Prototype
7. Adapter
8. Composite
9. Proxy
10. Flyweight

